

SpatialClaw: Rethinking Action Interface for Agentic Spatial Reasoning

Seokju Cho · Ryo Hachiuma · Abhishek Badki · Hang Su · Byung-Kwan Lee
· Chan Hee Song · Sifei Liu · Subhashree Radhakrishnan · Seungryong Kim ·
Yu-Chiang Frank Wang · Min-Hung Chen

ABSTRACT

*Spatial reasoning, the ability to determine where objects are, how they relate, and how they move in 3D, remains a fundamental challenge for vision-language models (VLMs). Tool-augmented agents attempt to address this by augmenting VLMs with specialist perception modules, yet their effectiveness is bounded by the action interface through which those tools are invoked. In this work, we study how the design of this interface shapes the agent’s capacity for open-ended spatial reasoning. Existing spatial agents either employ single-pass code execution, which commits to a full analysis strategy before any intermediate result is observed, or rely on a structured tool-call interface that often offers less flexibility for freely composing operations or tailoring the analysis to each task. Both designs offer limited flexibility for open-ended, complex 3D/4D spatial reasoning. We therefore propose **SpatialClaw**, a training-free framework for spatial reasoning that adopts code as the action interface. SpatialClaw maintains a stateful Python kernel pre-loaded with input frames and a suite of perception and geometry primitives, letting a VLM-backed agent write one executable cell per step conditioned on all prior outputs, enabling the agent to flexibly compose and manipulate perception results and adapt its analysis to both intermediate text and visual observations and the demands of each problem. Evaluated across **20 spatial reasoning benchmarks** spanning a broad range of static and dynamic 3D/4D spatial reasoning tasks, SpatialClaw achieves 59.9% average accuracy, outperforming the recent spatial agent by **+11.2 points**, with consistent gains across six VLM backbones from two model families without any benchmark- or model-specific adaptation.*

1 Introduction

[to main paper](#)

Spatial reasoning, the ability to determine where objects are, how they move, and how they relate in three dimensions, is foundational to visual perception. Questions such as “*Is the car moving toward the camera?*”, “*Which object is closest to the table?*”, and “*Did the person turn left or right?*” are effortless for humans, yet remain challenging for vision-language models (VLMs), including state-of-the-art models [5, 14, 29, 47]. Answering them requires composing

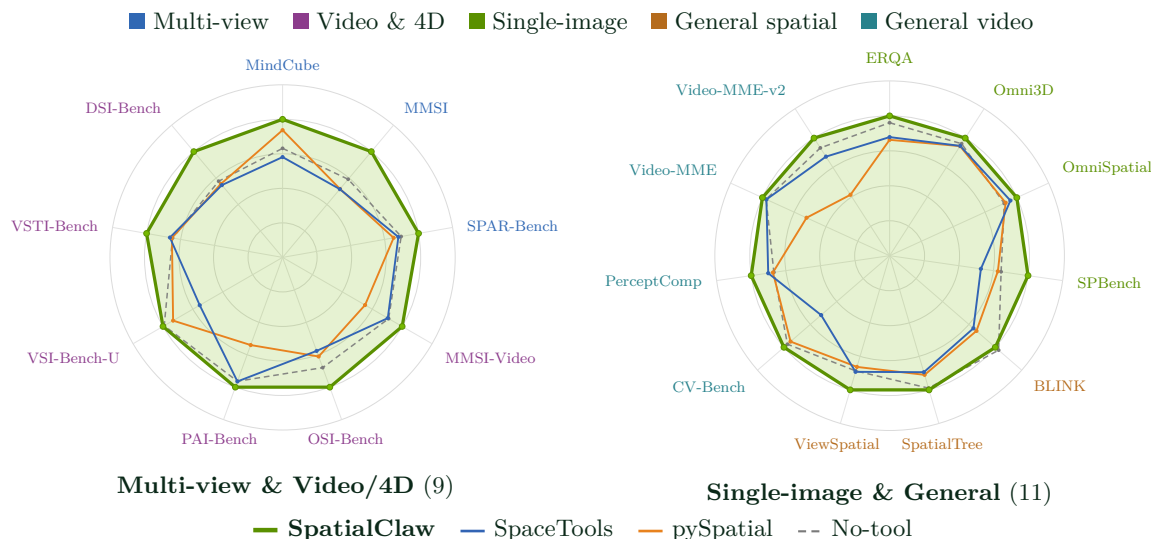


FIGURE 1 SpatialClaw improves spatial reasoning across the board. Per-benchmark accuracy on 20 spatial reasoning benchmarks (Gemma 4-31B backbone), split into two panels by task category. Each axis is individually rescaled so SpatialClaw traces the constant-radius ring. Baselines are SpaceTools-Toolshed [6], pySpatial [27], and a no-tool backbone.

multiple types of evidence, including depth, camera pose, and temporal correspondence, into a coherent geometric argument, and current VLMs struggle to perform this structured analysis reliably from pixels alone [4, 7, 8].

A natural response is to augment VLMs with specialist perception tools [6, 15, 16, 26, 34, 36], such as detectors [2], segmenters [3, 32], and depth or pose estimators [43], that produce perceptual outputs that VLMs would otherwise have to approximate from visual appearance alone. The capability of a tool-augmented agent, however, depends not only on which tools are available, but also on the *action interface* through which those tools are invoked. The action interface specifies how tools are invoked and their outputs represented, which intermediate states are observable between steps, and whether the agent can revise its reasoning in light of those observations before committing to an answer.

Prior tool-augmented spatial agents have primarily relied on one of two action interfaces. In *single-pass code execution*, the agent writes a complete Python program and runs it once [27, 28] (Fig. 2a), enabling flexible tool invocation but requiring a complete analysis strategy to be committed before any intermediate result has been observed. In *structured tool-calls*, the agent selects from a list of named tools, fills typed arguments, and consumes typed return values [6, 33, 49], offering limited support for composing and synthesizing tool outputs through external scientific libraries such as `numpy` or `scipy` to perform task-specific computations that emerge only at test time (Fig. 2b).

In practice, neither design readily supports the open-ended, compositional reasoning that complex 3D/4D spatial reasoning tasks often demand. We therefore argue that the appropriate action

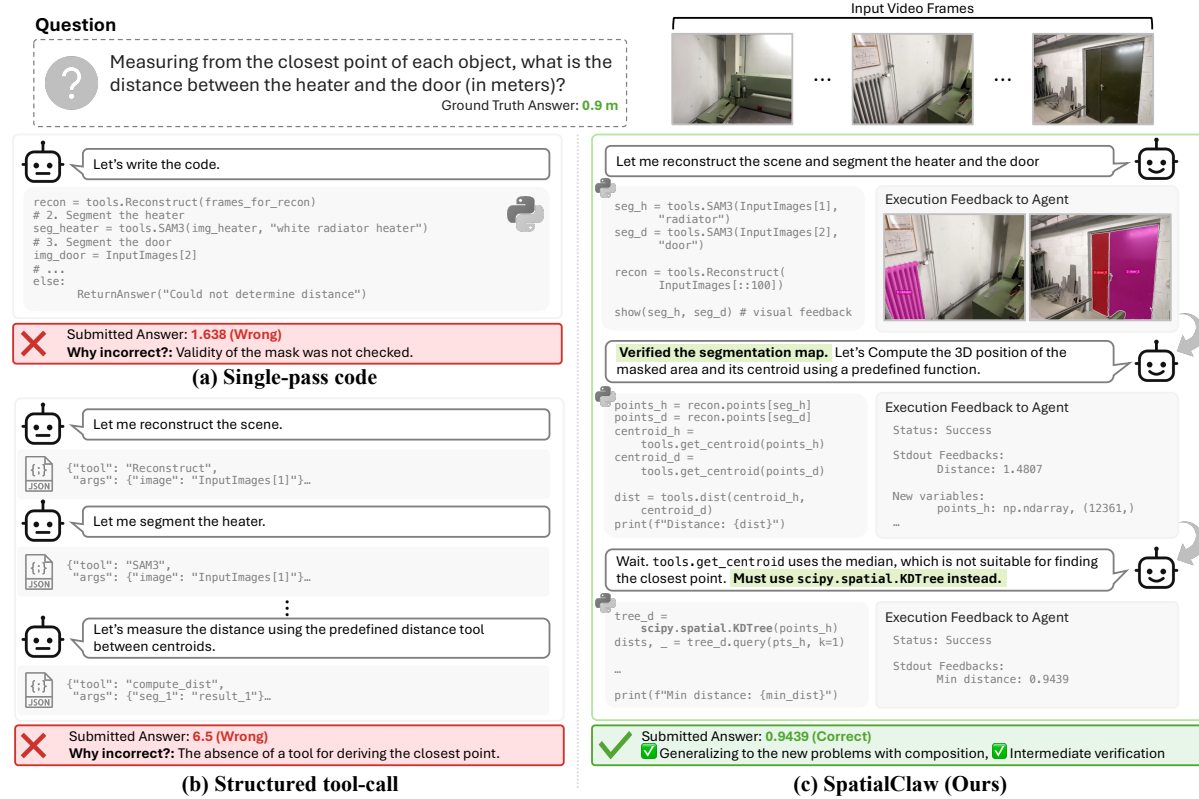


FIGURE 2 SpatialClaw studies code as the action interface for spatial reasoning. Three action interfaces on the same question. (a) A single-pass program chooses a complete computation before seeing its intermediate outputs. (b) A structured tool interface exposes common operations through structured commands (e.g., JSON, XML). (c) SpatialClaw writes Python in a persistent kernel, renders intermediate evidence, and revises the measurement before answering. In light of this, we propose **SpatialClaw**, a training-free framework that instantiates this principle as code as the action interface (Fig. 2c).

In SpatialClaw, perception tools are Python callables and their outputs, including masks, depth maps, camera geometry, and trajectories, are ordinary Python variables; the kernel preserves this state independently across turns, so any object produced at one step remains available for composition, inspection, and revision at all subsequent steps. While structured tool-calls and natural language reasoning can partially approximate this behavior, code generation provides a more expressive interface for adapting perception to the task, especially in spatial reasoning, where the required computations often cannot be anticipated by a fixed API. A new spatial analysis is not a new API entry, but a new composition of perception tools (e.g., segmentation [3] or reconstruction [24]) and numerical primitives (e.g., `numpy` [17], `scipy` [40]) assembled across steps in response to intermediate evidence. SpatialClaw pre-loads a persistent Python kernel with input frames, perception tools, and scientific libraries, and coordinates a VLM-backed agent through an iterative loop of planning, code execution, and feedback assembly, guided by a unified system prompt that encodes general principles of spatial reasoning rather than task-specific instructions.

We evaluate SpatialClaw on 20 spatial reasoning benchmarks spanning metric distance, camera and object motion, multi-view geometry, temporal reasoning, and spatial planning, to name a few. SpatialClaw achieves 59.9% average accuracy across all 20 benchmarks, outperforming the recent spatial agent [6] by **+11.2 points** (Fig. 1). Gains are largest on dynamic 4D video reasoning and multi-view inference, precisely the categories that demand chained geometric computation across frames and viewpoints, where no pre-specified tool call captures the required composition. Strikingly, these gains generalize out of the box: SpatialClaw delivers consistent improvements across the large majority of benchmarks and backbone models tested, spanning two distinct model families, Qwen [29–31] and Gemma4 [14], with parameters ranging from 27B to 397B, *without any modification* to the system prompt, tool set, or benchmark-specific engineering, confirming that the expressive action interface itself drives the gains rather than model-specific tuning. We further present a controlled comparison of three action interfaces and ablation studies, demonstrating that our interface achieves the strongest generalization through flexible composition of perception primitives and that the gains persist even when all pre-defined utility wrappers are removed.

Our contributions are as follows:

- **Rethinking the action interface for spatial reasoning agents.** We argue that the design of the action interface is a critical factor in agent capability in spatial reasoning, and introduce SpatialClaw, a training-free agent that instantiates this principle by replacing single-pass code execution and structured tool-call interfaces with a persistent, multi-turn Python kernel, turning each step into an opportunity to compose perception outputs with numerical primitives, inspect intermediate state, and revise the analysis.
- **Comprehensive evaluation that validates action interface design.** We establish an extensive evaluation spanning single-image spatial reasoning, multi-view spatial reasoning, general spatial reasoning, video spatial & 4D reasoning, and general video understanding, on which SpatialClaw achieves consistent improvement over baselines across most benchmarks and transfers across backbone models without any benchmark- or model-specific adaptation, providing a comprehensive empirical comparison of spatial agents.

2 Action Interfaces for Spatial Reasoning Agents

We characterize tool-augmented spatial agents by their *action interface*, that is, the medium through which they acquire, inspect, and transform visual evidence. When no interface is used, the VLM performs spatial reasoning by generating a chain-of-thought rationale in natural language directly over the input images [5, 14, 29, 47], without invoking any external computation or acquiring intermediate evidence. Beyond this tool-free case, prior spatial agents have primarily relied on one of two action interfaces: single-pass code execution, or predefined API calls through structured tool-calls.

Single-pass code. The agent writes a complete Python program, runs it once, and uses it to call perception modules and process their outputs [27, 28]. The program can express task-specific

computations, but it must commit to a complete strategy before observing any intermediate mask, depth map, plot, or runtime error. Retry mechanisms may repair a syntax or runtime failure, but intermediate visual evidence does not participate in the reasoning loop.

Structured tool-calls. The agent invokes perception modules by name, passing typed inputs and receiving typed outputs through predefined structured commands [6, 33, 49], e.g., JSON or XML. While this exposes a clean interface for invoking perception, compositions that can only be determined at test time, such as chaining a depth estimate with a segmentation mask in a way specific to the question, are difficult to express within the predefined command schema.

Motivation. These considerations motivate a third interface centered on code as a medium for flexible iterative composition based on intermediate evidence. Building on the rapid advances in code generation capabilities of modern LLMs, SpatialClaw lets the agent write one Python cell per step and execute it in a persistent kernel, conditioning each subsequent step on the resulting text, variables, errors, and visualizations; that is, we interpret code as the action interface. Code generation provides a more general mechanism for adapting to the task than either approach: computations such as finding the nearest object via `scipy.spatial.KDTree` or estimating a surface plane via RANSAC [10] emerge naturally from the problem at hand, without needing to be anticipated by a predefined interface. Spatial results such as depth maps, camera geometry, and temporal trajectories persist as ordinary Python variables across steps, remaining available for inspection, composition, and correction throughout execution. Fig. 2 provides a comparison of these three interfaces.

3 SpatialClaw

SpatialClaw realizes this code as the action interface through a persistent Python workspace for spatial reasoning. Rather than exposing a larger menu of spatial tools, it lets the agent turn visual evidence into inspectable spatial analyses that can be composed and revised over multiple steps. For each example, SpatialClaw creates a persistent Python kernel pre-loaded with the input frames and a set of primitives spanning perception modules (e.g., depth estimation, segmentation) and scientific libraries (e.g., NumPy [17], SciPy [40], Matplotlib [19]), and asks a VLM-backed agent to write one executable code cell at a time. Each cell can create masks, reconstructions, plots, or numeric summaries, and the resulting program state remains available to later cells. The next action is conditioned on text output, variable summaries, and rendered intermediate images from previous steps. The remainder of this section formalizes these two components: the persistent kernel workspace that maintains shared program state across execution steps (§3.1), and the outer agentic loop that coordinates planning, code generation, feedback assembly, and answer submission (§3.2).

3.1 Persistent Kernel Workspace

The workspace is initialized once for an example and discarded after the system terminates. It keeps all intermediate results as ordinary Python variables, so any object produced during

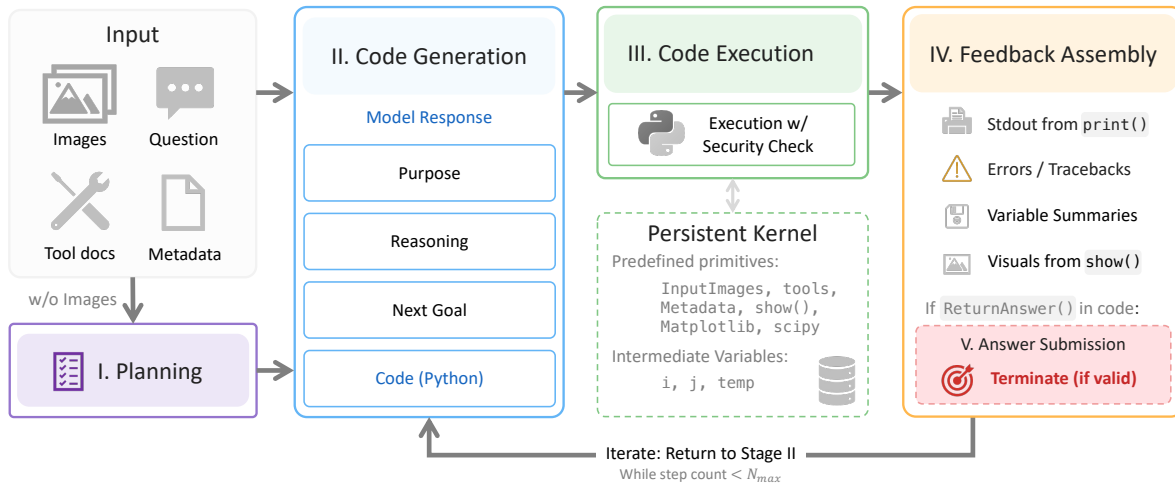


FIGURE 3 Agentic loop for iterative code execution. SpatialClaw wraps a persistent kernel in a five-stage loop. A planner receives the question and tool documentation but not the images, and produces an analysis plan. The main agent generates a Python cell executed in the persistent kernel. Feedback comprising stdout, variable summaries, and images registered via `show()` is appended to the model context. The loop continues until the agent submits an answer with `ReturnAnswer()` or the step count has reached the predefined maximum $N_{\max}$.

execution, including segmentation masks, depth maps, numeric arrays, and rendered plots, remains accessible to subsequent code cells. As a result, the agent does not need to choose the complete analysis in advance. It can start with a coarse computation, inspect the result, and refine the analysis against the observed evidence. The kernel exposes six public entry points.

1. `InputImages` contains the sampled frames or images.
2. `Metadata` contains frame rate, duration, and frame indices for video inputs, enabling the agent to reason about temporal structure when questions reference specific timestamps.
3. `tools` exposes evidence-producing perception and geometry primitives. `Reconstruct` wraps Depth Anything 3 [24] and returns per-frame depth, camera intrinsics, camera extrinsics, and dense point maps. `SAM3` [3] produces image or video masks from text, point, or box prompts. The `tools` module also includes lightweight utilities for mask operations, geometric computations, and visualizations. These utility functions simplify common operations such as mask manipulation and geometric computation, but the agent is not limited to what they provide; any operation can also be implemented directly over arrays or through the scientific libraries (`NumPy`, `SciPy`, `Matplotlib`) available in the execution environment. Full descriptions of all tools and utilities are provided in Appendix G.
4. `show(...)` registers an image to be embedded directly into the agent’s context at the next step, allowing the agent to visually inspect intermediate results such as masks, depth maps, or annotated frames before deciding what to do next. Figures rendered via `matplotlib` (`plt.show()`) are captured in the same manner and included in the agent’s next observation.

5. `vlm` dispatches queries to a separate VLM session, returning the response as plain text without embedding images into the main agent’s context. `vlm.locate(...)` performs visual grounding, returning bounding boxes. `vlm.ask_with_thinking(...)` handles questions that fall outside the perception toolset, such as those requiring commonsense reasoning or artistic style recognition. Further details on the prompts used for each VLM session are given in Appendix F.
6. `ReturnAnswer(...)` submits a candidate answer.

Together, these entry points form a self-contained computational environment: the agent reads observations through `InputImages` and `Metadata`, builds up spatial evidence via `tools`, inspects intermediate results through `show(...)`, queries a separate VLM session when needed via `vlm`, and commits a final answer through `ReturnAnswer(...)`. How the agent sequences these actions across multiple steps is governed by the outer loop described next.

3.2 Spatial Reasoning Loop: Code, Inspect, and Revise

The persistent kernel provides a runtime environment for the action interface, but does not by itself prescribe when to plan, when to act on the observed evidence, and when to commit to an answer. We therefore wrap the kernel in a five-stage outer loop (Fig. 3) covering planning, code generation, code execution, feedback assembly, and answer submission. The loop follows standard agent control structures and serves as a stable scaffold under which the construct, inspect, and revise pattern operates consistently across examples. The loop prescribes the order and termination conditions of each stage, but not how the agent should reason spatially within them; that reasoning discipline is encoded in the system prompt. Details of the agent system are provided in §E.

Disciplined reasoning via system prompt. `SpatialClaw` imposes structure on the open-ended Python action space through a unified system prompt, without any benchmark-specific engineering. The prompt defines both the runtime objects available in the kernel and the verification discipline the agent is required to follow. The prompt encourages the agent to treat spatial conclusions as claims that should be cross-checked against multiple evidence sources where possible. For instance, the agent is guided to resolve the relevant frame of reference, prefer metric computation over pixel-level impressions for geometric questions, and visually inspect tool outputs such as masks and annotated objects. It is also encouraged to sanity-check numerical magnitudes and consider any apparent disagreements between visualizations and computed quantities before invoking `ReturnAnswer()`. Because the prompt encodes general principles rather than few-shot examples or task-specific templates, the same configuration is applied across all benchmarks and backbone models without modification; more prompt details are given in §F. With this discipline in place, the loop proceeds through five stages as follows:

Stage I: Planning. The planner runs in a separate LLM session, isolated from the main agent’s execution context, with its own dedicated system prompt, and is invoked once per sample prior to execution. Unlike the main agent’s prompt, which focuses on general spatial reasoning

discipline, the planner’s prompt forbids executable code and pre-conclusion phrases, and instead instructs the planner to outline the analysis steps and the evidence to be acquired. It receives the question, metadata, and tool documentation, but not the input frames for efficiency. The resulting plan is appended to the main agent’s system prompt before execution begins, providing the agent with a structured plan to follow.

Stage II: Code generation. The main VLM-backed agent receives the question, the plan, the execution trajectory, and any images shown in previous steps. At each step, it produces a structured response comprising *purpose*, *reasoning*, *next goal*, and *code* fields, formatted in markdown. The *code* field is a Python cell that implements the next analysis action, which can call perception primitives, transform their outputs, render intermediate evidence, or submit an answer via `ReturnAnswer()`.

Stage III: Code execution. Before execution, a static checker parses the code’s abstract syntax tree (AST) to reject disallowed modules and unsafe builtins without running the code; on failure, the framework injects a concise traceback into the context and prompts the agent to revise the code. The validated cell is then executed in the persistent kernel.

Stage IV: Feedback assembly. The next model context receives the standard output produced by the executed cell (e.g., values printed via `print()`), concise tracebacks for any failed execution, summaries of newly created variables including their type, length, and size, and any images registered through `show()` for visual inspection. Because the kernel persists, later code can reuse earlier masks, reconstructions, plots, and partial analyses without recomputing them. The agent can therefore revise the analysis after seeing a wrong mask, an implausible trajectory, or a depth distribution that does not support the intended comparison. This feedback is appended to the model context, making intermediate results accessible at any subsequent step. The loop then returns to Stage II unless the agent has submitted an answer or the step count has reached the predefined maximum $N_{\max}$.

Stage V: Answer submission. When the agent determines that it has collected sufficient evidence, it submits an answer with `ReturnAnswer()`. The loop terminates once the submitted answer conforms to the expected format for the question type (e.g., multiple choice, numerical, or free-form text). Otherwise, the loop continues to the next step.

4 Results

Evaluation setup. We evaluate on 20 spatial reasoning benchmarks spanning single-image spatial reasoning [20, 28, 38, 46], multi-view spatial reasoning [41, 48, 50], general spatial reasoning [13, 22, 45], video spatial & 4D reasoning [1, 9, 25, 44, 53, 56], and general video understanding [12, 23, 39, 57]. Evaluation details are provided in §B. We evaluate across several open-source VLM backbones, including the 122B-A10B and 397B-A17B variants of Qwen3.5 [29], the 35B-A3B and 27B variants of Qwen3.6 [30], and the 31B and 26B-A4B variants of Gemma4 [14]. Due to the large number of benchmarks and backbones, we cap evaluation at 1,000 samples for benchmarks exceeding that size; otherwise, all samples are used. $N_{\max} = 30$ is

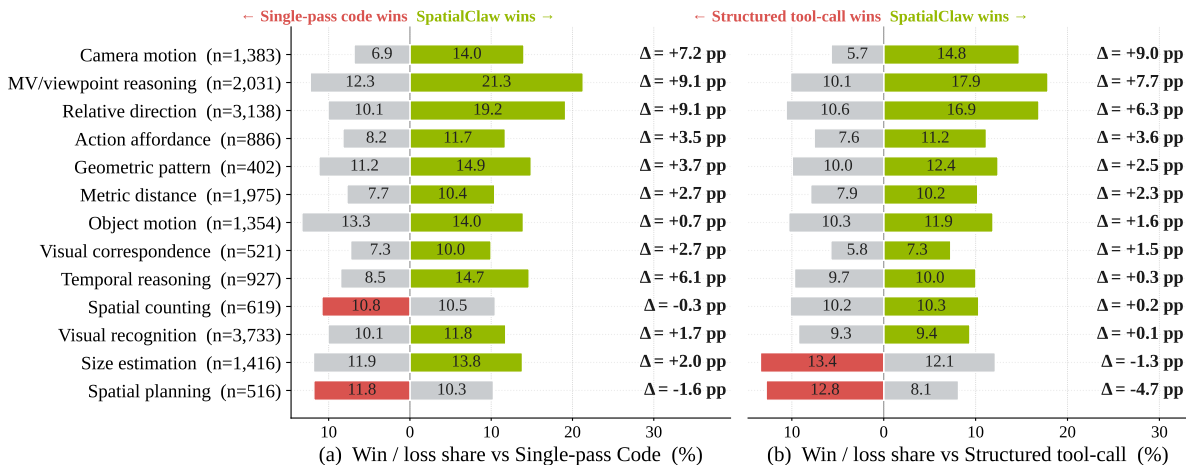


FIGURE 4 Pairwise win/loss margin of SpatialClaw over baselines across 13 meta-categories. SpatialClaw outperforms both (a) Structured tool-call and (b) Single-pass Code in 11/13 categories. The largest gains concentrate in categories that demand multi-step geometric composition.

used for all benchmarks. For all results in Tab. 1 and Tab. 2, the same hyperparameters, system prompts, and perception tools are used across all benchmarks and backbone models; further details are in the supplementary material.

Benchmark results. Tab. 1 reports results across 20 spatial reasoning benchmarks, spanning a broad range of tasks including video reasoning. SpatialClaw consistently improves over the no-tool baseline across all six backbone models, with the most pronounced gains on video spatial & 4D reasoning (e.g., DSI-Bench, avg. +18.3%p) and multi-view spatial reasoning (e.g., MindCube, avg. +14.3%p), categories that benefit most from iterative multi-step geometric computation across frames or viewpoints. Notably, these gains hold consistently across models ranging from 26B to 397B parameters without any modification, suggesting that our design generalizes to future models *without model-specific tuning*.

Action interface comparison. Tab. 2 compares SpatialClaw against two alternative action interfaces that share the same toolset and system prompt, differing only in the response format and the system prompt section describing the action interface. We also include a *No-tool* baseline that receives visual inputs and reasons directly without tool access. Implementation details for each baseline are provided in §D. *Single-Pass Code* moves to open code, but asks the model to generate one complete program before seeing execution feedback; *Structured Tool-Call* adds external perception through a JSON command interface. The results show that SpatialClaw consistently outperforms the other two action interfaces across all benchmarks, with the largest gains on tasks that require multi-step geometric composition. These results validate the generalizability of SpatialClaw’s action interface for spatial reasoning.

Quantitative comparison with other spatial agent methods. Tab. 3 compares SpatialClaw against recent spatial agent methods [6, 27, 28] using the same Gemma4-31B [14] backbone for

all methods, using the official implementation of each method. Here, VADAR and pySpatial fall into the category of single-pass code, while SpaceTools uses the structured tool-call interface. VADAR does not support video or multi-image inputs, so the corresponding entries are left blank.

Overall, SpatialClaw outperforms all baselines across all benchmarks, achieving the largest margin over SpaceTools (+11.2%p on average). Among the baselines, SpaceTools ranks highest, consistent with Tab. 2, which shows that structured tool-calls generally outperform single-pass code. SpatialClaw further improves over both action interfaces across all evaluated setups.

We also note that the baselines do not consistently improve over the no-tool baseline. For SpaceTools, we hypothesize that the method is designed to be fine-tuned via reinforcement learning, which may limit its zero-shot generalization. Other baselines may similarly have been optimized for narrower task categories, which could partly explain their limited performance across the diverse benchmarks evaluated here.

5 Analysis and Insights

In this section, we investigate *why* SpatialClaw’s action interface improves spatial reasoning beyond what structured alternatives can achieve.

Finding 0. SpatialClaw generalizes across diverse spatial reasoning tasks even without pre-defined utility tools.

Tab. 4 reports an ablation study examining how the composition of the tool set affects performance. In variant **(I)**, we remove all utility wrappers (e.g., `tools.Mask`, `tools.Geometry`) described in §G, retaining only the core perception tools (SAM3/DA3) and the scientific computing libraries (`numpy`, `scipy`) available in the execution environment. This configuration tests whether the agent can substitute pre-defined utility logic with on-the-fly numerical computation. Variant **(I)** achieves performance on par with full SpatialClaw, suggesting that the persistent kernel with scientific primitives can largely compensate for the absent utility tools. We also report a no-perception variant **(II)**, which removes the perception tools (SAM3/DA3) from full SpatialClaw, leaving only the code-as-action interface with scientific libraries. The resulting +2.7%p gain over the no-tool baseline isolates the contribution of the action interface itself, independent of the perception tools.

Finding 0. The agent spontaneously adapts its tool composition to the question type: distance questions preferentially invoke KD-tree search and norm operations, while direction questions rely on dot products. (Fig. 5)

To understand *how* the agent composes tools, we analyze the distribution of primitives (i.e., `numpy` and `scipy` operations) invoked across meta-categories (Figure 5). To enable a unified analysis across benchmarks, we group the fine-grained task categories of the 20 benchmarks into 13 semantically coherent meta-categories (e.g., depth estimation, relative direction, camera motion). The heatmap reveals a clear specialization: distance-type questions heavily use spatial

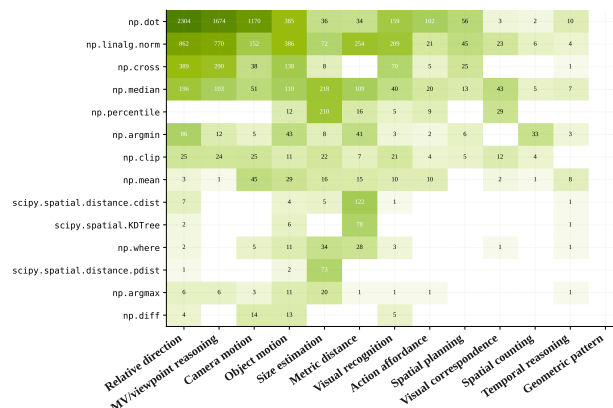


FIGURE 5 Composition adapts to the question type. Primitive usage frequency across meta-categories.

indexing (**KDTree**) and vector norms, which are the natural building blocks for nearest-neighbor and proximity; direction-type questions instead rely on dot products and angular operations, which capture orientation and heading. Critically, this specialization is *not* hard-coded; no category-specific prompt engineering or tool routing was applied. The agent selects geometrically appropriate primitives solely from the question semantics, demonstrating that SpatialClaw’s action interface unlocks a form of spontaneous, task-adaptive composition that is difficult to achieve with a structured tool-call interface.

Finding 0. SpatialClaw gains are largest precisely where chained geometric computation across frames and viewpoints is required, validating SpatialClaw’s action interface design. (Fig. 4)

In Figure 4, we compare SpatialClaw against the Structured tool-call and Single-pass Code baselines across 13 meta-categories. To construct these meta-categories, we map each benchmark’s fine-grained category labels into one of 13 coarser groups spanning all 20 evaluated benchmarks. SpatialClaw secures a net advantage in 11/13 categories over both Structured tool-call and Single-pass Code. The largest lifts (+6–9 pp) concentrate in Camera motion, Multi-view/viewpoint reasoning, and Relative direction, precisely the categories that require chained geometric computation across frames and viewpoints. This is where the persistent kernel, by enabling cross-step composition and revision, provides the greatest leverage. Where gains are smaller, the bottleneck is perception quality: Visual recognition tasks are already near-saturated by the backbone VLM, leaving little room for interface-level gains. Together, this breakdown confirms that the expressive action interface is the primary driver of performance, not model capacity or tool coverage.

6 Conclusion

We presented SpatialClaw, a training-free spatial reasoning agent that adopts code as the action interface, enabling a VLM to flexibly compose, inspect, and revise perception tool outputs across steps in a persistent Python kernel. Evaluated across 20 spatial reasoning benchmarks and six VLM backbones from two model families, SpatialClaw achieves 59.9% average accuracy and outperforms the recent spatial agent by **+11.2 points**, without any model- or benchmark-specific adaptation. These results demonstrate that the design of the action interface is a highly impactful yet underexplored axis of improvement for spatial reasoning agents.

Acknowledgements

We are deeply grateful to Valts Blukis, Siyi Chen, Yi Dong, Tsung-Yi Lin, Karan Sapra, Andrew Tao, Bowen Wen, and Zhiding Yu for their valuable discussions, insightful feedback, and generous support throughout this work.

Supplementary Material

.toc	appendix	mainpaper	none	appendix	subsection	subsection	section	
subsection								

Contents

Contents

1	Introduction	1
2	Action Interfaces for Spatial Reasoning Agents	4
3	SpatialClaw	5
3.1	Persistent Kernel Workspace	5
3.2	Spatial Reasoning Loop: Code, Inspect, and Revise	7
4	Results	8
5	Analysis and Insights	10
6	Conclusion	12
	Supplementary Material	12
A	Related Works	13
B	Evaluation Protocol	14
C	Additional Analysis	15
D	Baseline Implementations	16

D.1 Single-Pass Code	16
D.2 Structured Tool-Calls	17
<i>E Agent System Design</i>	17
E.1 System Configuration	17
E.2 Input Preprocessing	18
E.3 Persistent Kernel	18
E.4 Security Sandbox	18
E.5 Per-Frame Type Contract	18
E.6 Error Handling	19
E.7 Backbones	20
<i>F Prompt Details</i>	20
F.1 Main Agent Prompt	21
F.2 Planner Prompt	22
F.3 Vision Prompts	22
<i>G Tool API Reference</i>	23
G.1 <code>tools.Reconstruct</code>	23
G.2 <code>tools.SAM3</code>	24
G.3 <code>tools.Geometry</code>	26
G.4 <code>tools.Mask</code>	27
G.5 <code>tools.Time</code>	29
G.6 <code>tools.Graph</code>	30
G.7 <code>tools.Draw</code>	32
<i>H Limitations and Broader Impact</i>	35
H.1 Limitations	35
H.2 Broader Impact	39

A Related Works

Spatial reasoning in VLMs. Despite broad progress in vision-language models (VLMs), spatial reasoning remains a persistently limited capability [4, 7, 35]. A common response is to fine-tune VLMs with spatial supervision, either by distilling 3D annotations into instruction data [4, 7] or by augmenting the model with explicit geometry modules [9, 18, 51]. These approaches produce fast, self-contained models, but require retraining whenever perception modules or task distributions change.

Tool-augmented visual agents. Tool-augmented visual agents have an LLM compose calls to specialist vision modules [15, 26, 34, 36, 54]. Early work synthesizes a full program in a single pass [15, 36], subsequent work dispatches requests through structured tool menus [26, 34]. These systems establish that external perception can extend an LLM beyond its native visual capabilities, but they either constrain the agent to a fixed tool interface that may limit generalization to novel compositions, or commit to a full program before any intermediate output can be inspected, or fuse perception and planning inside a single multimodal model that cannot independently verify intermediate tool outputs.

Spatial reasoning agents. GCA [49] decouples the VLM into a semantic analyst that formalizes the query as geometric constraints and a task solver that executes tool calls within those bounds, RieMind [33] grounds an LLM in an explicit 3D scene graph queried through typed geometric operations, and SpaceTools [6] fine-tunes a VLM to coordinate a predefined set of perception tools through supervised demonstrations followed by interactive reinforcement learning. Think3D [52] uses a closely related but more specialized interface, repeatedly selecting camera viewpoints to render a reconstructed point cloud and reasoning over the rendered images. pySpatial [27] composes reconstruction, camera-pose recovery, and novel-view synthesis into a 3D visual program, and VADAR [28] first synthesizes a task-specific Pythonic API and then a program that calls into it. Neither writes code turn-by-turn in response to intermediate execution results, so per-step inspection of perception outputs is limited.

Code action interface for LLM agents. CodeAct [42] showed that emitting executable Python code as the action outperforms JSON- and text-formatted action spaces for general-purpose LLM agents. SpatialClaw instantiates this paradigm for spatial reasoning, contributing domain-specific design choices absent from general-purpose frameworks. Where CodeAct focuses on the infrastructure of code execution, our contribution is the spatial-reasoning angle, namely controlled comparisons against alternative action interfaces and trace-level analyses that identify when code action interface helps and which spatial analysis patterns drive the gains (§4, §5).

B Evaluation Protocol

We evaluate SpatialClaw on the 20 spatial reasoning benchmarks reported in the main results, organized into five categories: single-image spatial reasoning (ERQA [38], Omni3D [28], OmniSpatial [20], SPBench [46]), multi-view spatial reasoning (MindCube [41], MMSI [48], SPAR-Bench [50]), video spatial & 4D reasoning (MMSI-Video [25], OSI-Bench [44], PAI-Bench [56], VSI-Bench-U [1], VSTI-Bench [9], DSI-Bench [53]), general spatial reasoning (BLINK [13], SpatialTree [45], ViewSpatial [22]), and general video understanding (CV-Bench [57], Percept-Comp [23], Video-MME [12], Video-MME-v2 [39]). Table 5 reports the per-sample scoring metric applied to each benchmark.

We apply a consistent per-sample scoring protocol across all benchmarks, using the same metric family while respecting each benchmark’s original threshold settings. Categorical questions, including multiple-choice and binary questions, receive a score of 1 when the predicted answer matches the reference and 0 otherwise. Numerical questions are scored with mean relative accuracy (MRA) [47], where the acceptance threshold follows each benchmark’s originating implementation. SPAR-Bench’s view-change-inference task uses VCI, defined as MRA averaged over its five movement axes. The reported number for each benchmark is the unweighted mean of the per-sample scores.

For benchmarks with more than 1,000 samples, we evaluate a randomly chosen subset of 1,000 samples drawn with a fixed seed; the same subset is reproduced by running the supplementary code with the same seed.

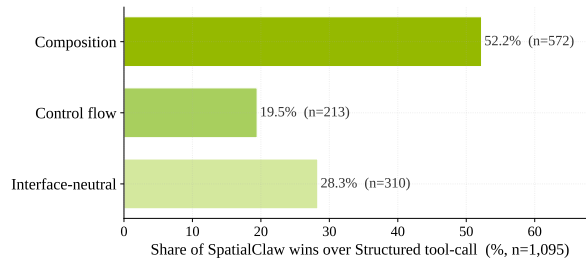


FIGURE 6 Attribution of SpatialClaw’s wins over structured tool-call via LLM-as-judge. Over half of the gains are driven by *code composition*, 19.5% by *control flow*, and 28.3% are *interface-neutral* wins on perceptual tasks unaffected by the action interface.

C Additional Analysis

Finding 0. Composition is the main driver of SpatialClaw’s gains over structured tool-call. (Fig. 6)

To identify the main driver of SpatialClaw’s gains over structured tool-call baseline, we examine the instances where SpatialClaw answers correctly but structured tool-call fails (Figure 6). For each such instance, we provide an LLM judge (Gemini-3.1-Pro [37]) with the full reasoning traces of both systems, the ground-truth answer, and the input images, and ask it to assign a binary label to each predefined attribution category; instances in which all categories receive a negative label are classified as *interface-neutral*.

We find that over 50% of the instances are attributed to *code composition* (i.e., chaining multiple tool calls into a single coherent program), while 19.5% are attributed to *control flow* (e.g., *if* or *for* statements that conditionally branch or iterate over intermediate results). The remaining 28.3% are *interface-neutral* wins, in which the correct answer depends on visual recognition or luck rather than the agent’s action space, and either interface would have been equally capable. This is further causally supported by Tab. 4, which shows that performance degradation is minimal even when predefined utility functions are removed and replaced with on-the-fly numerical computation.

Finding 0. Geometric reasoning errors constitute a leading failure mode, with perception errors driven by VLM hallucinations and tool limitations as a notable secondary contributor.

In Fig. 7, we analyze the agent’s failure modes by categorizing 1,000 incorrect samples with LLM-as-Judge [55], using a strong proprietary model (Gemini-3.1-Pro). For each sample, the judge is provided with the agent’s full reasoning trace and the ground-truth answer, and assigns each session to one of 11 human-designed fine-grained categories.

On the perception side, the agent frequently attempts visual judgments that exceed the capabilities of its available tools, and its underlying VLM occasionally hallucinates objects, attributes, or spatial relations that are not actually present in the scene. Lower-level perception primitives

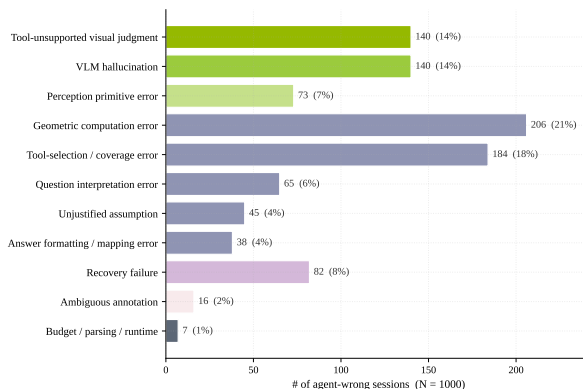


FIGURE 7 Failure-mode breakdown of incorrect agent sessions. Each session is classified by an LLM-as-Judge (Gemini-3.1-Pro [37]) into one of 11 fine-grained failure categories.

such as detection and segmentation also produce localized errors that propagate into downstream reasoning. The predominant reasoning failures are geometric in nature, as the agent frequently errs in handling 3D coordinates, distances, angles, or projective relationships, even when the underlying perceptual evidence is sound. Closely related is a class of tool-selection and coverage failures, in which the agent invokes an inappropriate tool or omits a necessary intermediate step. We additionally find cases of question misinterpretation, unjustified assumptions about unobserved scene properties, and inconsistencies between the internal computation and the final answer format. Beyond these direct errors, a non-trivial fraction of failures arises from the agent’s inability to recover upon detecting an inconsistency, as it either commits to a flawed early hypothesis or oscillates without converging. The remaining residual cases involve ambiguous ground-truth annotations and occasional budget, parsing, or runtime issues, which together account for only a small share of observed failures.

A promising direction for future work is to apply reinforcement learning to improve tool selection, coding of geometric operations, and error recovery within the agent loop.

D Baseline Implementations

§2 of the main paper compares the code as the action interface of SpatialClaw against two baselines, single-pass code and structured tool-calls. To isolate the effect of the interface, both baselines share the agent loop, the persistent kernel, the perception tools, and the planner; only the format of the per-step action differs, along with any adjustment to the per-sample step budget necessitated by the format.

D.1 Single-Pass Code

The single-pass baseline collapses the multi-step loop to a single iteration. The agent receives the question, the key frames, and the same main-agent system prompt, and must produce one Python cell that calls perception tools, performs any required computation, and submits the answer

through `ReturnAnswer`. Planning is disabled, and the workflow section of the system prompt is replaced by a single-turn variant that informs the agent there will be no further opportunity to act. The agent must therefore commit to a complete strategy before observing any intermediate mask, depth map, plot, or runtime error. When the cell does not reach `ReturnAnswer`, the same termination fallback used by the code as the action interface variant (§E.6) returns a best-effort answer.

D.2 Structured Tool-Calls

The structured-tool-call baseline keeps the multi-step loop intact but constrains each step to a single named tool invocation expressed as JSON. The *Code* section of the response envelope is replaced by a *Tool Call* section that contains exactly one object of the form

```
1 {"tool": "tools.SAM3.segment_image_by_text",
2  "args": {"image": "InputImages[0]", "prompt": "red car"}}
```

The specified tool call is first translated to Python code and then executed on the same kernel used by SpatialClaw, and its return value is stored in a named variable that later steps can reference. Arguments must be either literal values or references to previously bound names (i.e., the input variables, the tool catalogue, or the outputs of prior steps). Arbitrary Python expressions are not allowed, and each step issues exactly one tool call, and intermediate results are accessible only through these named per-step variables.

E Agent System Design

E.1 System Configuration

SpatialClaw runs on two independently scalable serving roles. The first role hosts the language-model backbone and serves every LLM call the system makes, including the main agent, the planner, and the two isolated visual sessions invoked from inside the kernel (`vlm.locate` for coordinate grounding and `vlm.askwiththinkingforvisualreasoning`). *The backbone is served via vLLM [21]; multiple endpoints are compatible interface, and a session-aware router dispatches each call so that the prefix of the conversation lands on a consistent*

The second role hosts the perception models behind a lightweight HTTP service: Depth Anything 3 [24] for 3D reconstruction and SAM3 [3] for segmentation. The kernel-side tools `tools.Reconstruct` and `tools.SAM3` are light-weight clients that call this service and convert the results into ordinary numpy arrays before returning to the agent. For Depth Anything 3, we use the `DA3Nested-Giant-Large` variant, which pairs an any-view giant model with a metric depth model to reconstruct visual geometry at real-world metric scale.

This decomposition has two practical consequences. First, the language-model and perception backbones scale independently: the number of LLM nodes and perception nodes can each be tuned to their respective call demand without any changes to the agent code. Second, all LLM roles (agent, planner, grounding, and thinking) share the same backbone pool, so the main conversation and in-kernel visual queries share the same prefix cache and benefit from KV reuse.

E.2 Input Preprocessing

All input images are resized such that the long edge does not exceed 768 pixels, preserving the original aspect ratio; this bounds the per-frame token cost. The agent VLM receives at most 32 frames as visual context: for video inputs longer than 32 frames, frames are drawn at uniform temporal intervals to yield 32 key frames; for multi-image inputs, the sequence is truncated to the first 32 images. The persistent kernel retains the full sampled frame sequence, allowing any frame to be revisited via `show()` or forwarded to a perception tool in subsequent steps. When a sample contains multiple videos, each is maintained as an independent frame sequence, enabling the agent to analyze and compare them independently.

E.3 Persistent Kernel

Each example is solved within a dedicated, stateful IPython kernel that persists throughout the entire inference run for that sample. Variables created during one cell (segmentation masks, reconstructions, depth and point arrays, intermediate visualizations, and partial numerical results) remain in scope for every subsequent cell, so the agent can revisit and recompose earlier evidence without re-issuing the underlying tool call. A per-cell wall-clock timeout protects the system from runaway code, and on repeated execution failures the kernel is restarted; when this happens, the input frames, sample metadata, the `tools` module, the `feedback` module, and `ReturnAnswer` are automatically re-injected so the agent resumes against an identical environment.

E.4 Security Sandbox

Code emitted by the LLM is statically analyzed before execution. The analyzer traverses the Abstract Syntax Tree (AST) of each cell to identify and reject unsafe patterns, including file I/O, network access, dynamic-code primitives such as `exec` and `eval`, and direct imports of GPU-backend libraries. A complementary regular-expression pass catches patterns that may evade AST traversal, such as method-style writes (`.save`, `.to_csv`). When a cell is rejected, the corresponding error message is delivered to the agent as feedback so it can revise the code within the same step, without adding latency to steps that pass inspection.

E.5 Per-Frame Type Contract

Every frame-indexed perception output is encapsulated in a typed container that records the absolute frame indices to which it corresponds. Composing two such containers (e.g., a per-frame segmentation mask with a per-frame point map) triggers an automatic frame-index alignment check at the composition site; index mismatches raise an immediate exception identifying the offending frames. This mechanism guards against a class of silent semantic errors in which tensors are numerically well-formed but spatially misaligned, such as applying a mask derived from one frame to the point map of another. By surfacing the error at the point of composition, the violation appears directly in the agent’s subsequent feedback rather than propagating silently to produce an incorrect final answer.

E.6 Error Handling

Adopting code as the action interface exposes the system to a broader class of runtime failures than a fixed-API interface. SpatialClaw treats each such failure as an additional observation by routing the resulting exception or traceback directly into the agent’s next feedback message, enabling the agent to diagnose and revise its code within the same episode rather than terminating on error.

Response-format errors. If the agent’s response cannot be parsed into the four required fields (*Purpose*, *Reasoning*, *Next Goal*, *Code*), the validator records a *format error* for the step. The malformed text is replaced in the conversation with a short placeholder that names the violation, and the agent is reminded to re-read the required format on the next turn. The malformed body is never echoed back, both to keep the context clean and to discourage the agent from imitating it.

Sandbox rejections. Code that the security analyzer rejects (§E.4) is never executed. Instead, the rejection reason (identifying the forbidden module, builtin, or pattern) is returned as the step’s error and surfaced to the agent in the next feedback message, exactly as if the code had failed at runtime. The agent therefore retries the same step with revised code rather than escalating.

Cell-execution exceptions. When a cell raises a Python exception, three steps are applied to trim the error output before the failure is forwarded to the agent. The following example illustrates the condensed representation that is inserted into the conversation history after a faulting step:

```

1  **Purpose**: Perform 3D reconstruction and extract point clouds for the fireplace and the painting.
2  **Reasoning**: [errored -- condensed]
3  **Next Goal**: [errored -- condensed]
4  **Code**:
5  \begin{lstlisting}
6  # 1. 3D Reconstruction
7  recon = tools.Reconstruct.Reconstruct(InputImages)
8
9  # 2. Extract point clouds
10 # Fireplace is in InputImages[1]
11 fi_fire = seg_fireplace.frame_indices[0] # <-- ERROR
12 # NameError: name 'seg_fireplace' is not defined

```

First, the traceback is truncated to the final exception type, its message, and the offending source line; surrounding interpreter frames that reflect the execution infrastructure rather than the agent’s code are discarded. Second, any code appearing after the faulting line is excluded from the context window, since it was never executed and therefore contributes no evidence to the agent’s subsequent reasoning. Third, the *Reasoning* and *Next Goal* fields of the failed step are replaced by a condensed summary prior to insertion into the conversation history, preventing a single failure from disproportionately consuming the available context budget.

Cell timeouts. Each cell is subject to a per-cell wall-clock timeout to guard against non-terminating or excessively long-running code. Upon timeout, the kernel’s user namespace is

cleared and the execution environment is restored by re-injecting the input frames, sample metadata, and the `tools`, `feedback`, and `ReturnAnswer` entry points; the agent receives a timeout notification in its next observation and is expected to revise its code accordingly. If namespace restoration fails, the kernel process is fully restarted and the same re-injection sequence is applied.

Perception-tool retries. Calls to the GPU-backed perception tools may fail transiently due to server restarts or temporary overload. Each `tools.Reconstruct` or `tools.SAM3` call is automatically retried up to a fixed number of times, with increasing wait intervals between attempts; each retry also re-selects an available endpoint to route away from an unhealthy server. Only if all retries are exhausted is the error propagated to the agent as a Python exception. Similarly, transient network errors on the language-model side (e.g., connection drops or gateway timeouts) are distinguished from agent-side failures and do not count against the agent’s error budget.

Hard budgets and forced termination. The loop carries explicit budgets on the number of LLM steps, the number of consecutive step-level failures, and the cumulative number of tool calls. When any budget is exceeded, the loop hands control to a termination node that is guaranteed to return an answer. The termination node attempts two fallback strategies in order: (i) a chain-of-thought fallback that prompts the VLM backbone to answer directly from the key frames using the same boxed-answer protocol as the no-tool baseline; and (ii) if the chain-of-thought fallback does not yield a parseable answer, a regex-based extraction over the agent’s recent messages and variables. The agent therefore always returns a best-effort answer for any sample whose execution started, even when the persistent-kernel path was unable to reach `ReturnAnswer` on its own.

E.7 Backbones

We evaluate SpatialClaw with the six open-source backbones listed in Table 6. All six are served through the same vLLM-based serving role described in §E.1. Crucially, every backbone is evaluated under *identical* configuration: the same system prompt, the same set of perception tools, the same maximum step count, and the same input preprocessing. No prompt template, tool subset, or budget is tuned per benchmark or per backbone.

F Prompt Details

SpatialClaw defines four VLM roles, each with its own system prompt: the main agent that writes code, the planner that produces a plan before any image is observed, and two isolated visual sessions invoked from inside the kernel through `vlm.locate` (coordinate grounding) and `vlm.ask_with_thinking(visualreasoning)`. *The number of calls per role varies per sample : the main agent is queried once per step, the planner is invoked once per sample, and the two kernel – invoked sessions are recalled as many times as the agent decides. This appendix summarizes what each prompt encodes.*

F.1 Main Agent Prompt

The main agent’s system prompt is organized into a fixed sequence of sections, which we describe in order. A short header introduces the role of the agent and names the input variables (`InputImages` and `Metadata`).

Response format. The first content section declares the response format: every reply must contain four markdown sections with the headings *Purpose*, *Reasoning*, *Next Goal*, and *Code*, where the code field holds a single Python cell to be executed in the kernel.

Visual access. The two following sections document the visual access surface available inside the kernel. The `show()` entry displays one or more images inline in the next feedback message, notes that `matplotlib` figures are auto-captured, and states the per-session image budgets. The `vlm.locate` entry calls an isolated grounding session that returns coordinates in a 0–1000 normalized scale and accepts up to eight images per call, and the `vlm.askwithhinkingentry` calls an isolated reasoning session that returns a textual answer over up to 64 frames. The same session

Available tools. The *Available Tools* section then documents the seven tool namespaces that the kernel exposes, with method signatures, argument types, expected input shapes, and short usage examples for each: `tools.Reconstruct`, `tools.SAM3`, `tools.Graph`, `tools.Time`, `tools.Mask`, `tools.Geometry`, and `tools.Draw`. The details can be found in §G.

Coordinate systems. The coordinate-system section names four reference frames (pixel space, camera space, world space, and an object-centric frame defined by an object’s facing direction), states the technical convention for camera-to-world matrices returned by `tools.Reconstruct` (4×4 SE(3) matrices in OpenCV convention, with gravity-aligned world axes anchored to the first camera), and gives a code snippet that computes left/right/front/behind classifications by projecting the world-frame vector to a target onto the camera’s forward and right axes.

Robust computation. The robust-computation section lists five principles: prefer median over mean for aggregations, compare across multiple frames before drawing conclusions, reason in metric units rather than pixels, sanity-check magnitudes against physical priors, and print numerical values before committing to a conclusion.

Cross-validation. The cross-validation section enumerates four complementary evidence sources (visual perception via `show()`, geometric computation, visualizations, and logical reasoning) and specifies a five-step diagnostic procedure that the agent follows when two of them disagree.

Kernel-side contract and budget. The remaining sections fix the kernel-side contract and the run-level budget. The `ReturnAnswer` section documents the single-call API used to submit a final answer and the accepted argument types (`str`, `int`, `float`). The code-rules section lists the modules pre-imported into the kernel, the modules and built-ins forbidden by the security sandbox, and the reserved names that the agent must not reassign (`feedback`, `tools`, `InputImages`, `Metadata`, `ReturnAnswer`, `show`, `RefImages`). The workflow section instructs the agent to follow the planner’s plan and to confirm any spatial conclusion against at least two independent lines of evidence before calling `ReturnAnswer`, and is followed by a single line

stating the per-sample step budget. A final session-input section describes the per-sample input variables, the frame-indexing convention for `InputImages`, the fields of `Metadata`, and the key-frame-to-variable mapping; an additional reference-images section is included when the sample contains inline `[reference image #N]` tags.

F.2 Planner Prompt

The planner is invoked once per sample, before the main agent begins execution. The planner receives the question text, the per-sample metadata (frame count, frame indices, and frame-rate fields), and the system prompt only.

Shared documentation. After a one-paragraph header that declares the planner’s role and states that it does not see the actual frames, the prompt repeats the same *Available Tools*, `show()`, `vlm.locate`, `vlm.askwiththinking, and coordinate – system documentation that the main agent receives, and embeds the cross – validation and robust – computations sub – sections from the main agent prompt.`

Planning strategy. The planning-strategy section maps question shapes to tool choices: questions about object coordinates point to `vlm.locate` followed by `tools.SAM3`; questions about three-dimensional geometry, depth, or camera pose point to `tools.Reconstruct`; questions answerable by visual judgment point to `show()`; and questions that require a textual reading over several frames, or a fallback when a specialized tool fails, point to `vlm.askwiththinking`. *The same section gives explicit guidance on annotation overlays, on coordinate grounding through 0–1000 normalized pixel coordinates, and on the choice between quantitative computation and qualitative visual reading.*

Task structure and rules. The session-input section reports the frame count and the frame-indexing convention and states that the planner cannot see the images, instructing it to plan an investigation rather than answer the question from the question text. The task section asks for six items in the output: a task analysis (including an explicit choice of coordinate system when the question is ambiguous), a list of information needs, an ordered computation plan, a verification checklist, a list of cross-validation steps, and a fallback plan. The critical-rules section states that the planner must respond in plain text only, must not include JSON outside the verification checklist, must not write executable Python code, and must not produce the answer or pre-conclude phrases such as “the answer is likely ...” or “I expect the answer is ...”; the planning-strategy section adds that any hypothesis the planner forms from the question is itself what the tools must verify, rather than evidence in its own right.

F.3 Vision Prompts

The two isolated visual sessions invoked from the kernel use their own system prompts.

Grounding session (`vlm.locate`). The grounding session called by `vlm.locate` specifies a three-step procedure: identify the exact object, annotation, or marker that the question describes; classify the image as `PRESENT`, `ABSENT`, or `AMBIGUOUS` with respect to that description; and answer accordingly. `PRESENT` cases produce coordinates in the format requested by the

calling question. ABSENT and AMBIGUOUS cases reply with the literal string **Not visible** on its own line, optionally followed by one short clarifying sentence. The prompt includes three worked examples that illustrate each of the three classifications, and ends with a short formatting block stating that coordinates use the 0–1000 normalized scale, that the assistant follows a “Reply with ONLY the numbers” instruction when present, and that the same prompt also covers segmentation-overlay assessment and plot-and-chart reading.

Reasoning session (`vlm.ask_with_thinking`). The reasoning session called by `vlm.ask_with_thinking` specifies five rules : 1) ground every claim in what is directly observable in the supplied images, 2) use a line notewhen the supplied frames do not constrain the answer, and 5) treat each call as independent of any prior context.

G Tool API Reference

This appendix expands the one-paragraph tool description in the main paper into per-tool signatures and behaviors. The first two tools (`tools.Reconstruct` and `tools.SAM3`) are thin clients that call the perception serving role described in §E.1; the remaining tools are pure-Python helper functions that execute in the same process as the kernel.

G.1 `tools.Reconstruct`

`tools.Reconstruct` is a client wrapper for the perception serving role described in §E.1. Its single entry method calls Depth Anything 3 [24] on a frame batch and returns a **Reconstruction** object that exposes per-frame depth, camera intrinsics, camera-to-world extrinsics, a dense per-frame point map in world coordinates, and a top-down rendering helper function. The output coordinate system is gravity-aligned with respect to the first camera: $+Y$ points up, the ground plane sits at $Y \approx 0$, and the first camera looks toward $-Z$. The output spatial resolution matches the input frame resolution, so masks from `tools.SAM3` can be combined directly with the reconstructed point maps without resizing.

Method.

- `Reconstruct(frames, frame_indices = None)`
- *Input:* `frames` is a list of `FrameImage` or `PIL` (Python Imaging Library) images, with at most `reconstruct_max_frames_entries` (set to 64 in our experiments). When `FrameImage` objects are passed, absolute video frame index is extracted from each frame’s `frame_index` attribute and the `frame_indices` argument may be omitted. *Output:* a **Reconstruction** object.

Reconstruction attributes. The returned object is indexed by absolute frame index, not by position in the input list.

- `frame_indices` : `list[int]`, absolute frame indices in the same order as `frames.num_frames` : `int`; `metric_scale` : `float`.
- `depth[fi]`: (H, W) `float32` depth map at absolute frame `fi`.

- `extrinsics[fi]`: (4, 4) float64 camera-to-world SE(3) matrix in OpenCV convention.
- `intrinsics[fi]`: dictionary with keys `fx`, `fy`, `cx`, `cy`.
- `points[fi]`: (H, W, 3) float32 dense point map in world coordinates.

`render_bev.`

- `recon.render_bev(masks = None, labels = None, ref_frame = 0, ego_trajectory = True)`
 - *Input*: `masks` is a `PerFrameMask` or an (N, N_{obj} , H, W) boolean array; `labels` provides per-object names when a raw array is passed; `ref_frame` is the absolute frame index of the reference camera; `ego_trajectory` toggles drawing of the camera path.
 - *Output*: a `VisualFeedback` that the agent inspects with `show()`. Stationary objects are annotated with oriented bounding boxes and moving objects with colour-graded trajectory lines.

G.2 tools.SAM3

`tools.SAM3` is a client wrapper for the perception serving role and exposes single-frame and video-mode segmentation against SAM3 [3], together with an object-existence check. All segmentation methods return a `PerFrameMask` whose frame indices are absolute video frame indices and align with the indices used by any `Reconstruction` the agent has built.

Image-mode methods. Each operates on a single image and returns a `PerFrameMask` with one frame.

- `segment_image_by_ext(image, prompt, label = None)`
 - *Input*: `image` is a PIL image; `prompt` is a free-form text description.
 - *Output*: `PerFrameMask` with one channel per detected instance.

`segment_image_by_points(image, points, point_labels, label = "object")`

Input: `points` is a list of [x, y] pixel coordinates and `point_labels` is a parallel list with values 1 (foreground) or 0 (background).

Output: `PerFrameMask` for the prompted object.

`segment_image_by_box(image, box, label = "object")`

Input: `box` is [x1, y1, x2, y2] in pixels (xyxy convention).

Output: `PerFrameMask` for the boxed object.

Video-mode methods. Each propagates the segmentation across the requested frame range, with at most `sam3_max_video_frames` frames per call. `start_frame` and `end_frame` are absolute video frame indices that select a temporal window;

`prompt_frame_idx` is local to that window; `video_index` is 1-indexed and selects the source video in multi-video samples.

- `segment_video_by_ext(prompts, labels = None, prompt_frame_idx = 0, start_frame = None, end_frame = None, video_index = 1)`
 - *Input:* `prompts` is a list of text descriptions, one per object.
 - *Output:* `PerFrameMask` over the selected frame range.

`segment_video_by_points(points_per_object, point_labels_per_object, labels, prompt_frame_idx = 0, start_frame = None, end_frame = None, video_index = 1)`

Input: per-object point clicks on the prompt frame, parallel foreground/background labels, and human-readable per-object labels.

Output: `PerFrameMask` over the selected frame range.

`segment_video_by_box(boxes, labels, prompt_frame_idx = 0, start_frame = None, end_frame = None, video_index = 1)`

Input: per-object bounding boxes on the prompt frame and parallel human-readable labels.

Output: `PerFrameMask` over the selected frame range.

Object-existence check.

- `is_object_exist(images, object_name)`
 - *Input:* `images` is a list of PIL images and `object_name` is a text description.
 - *Output:* a dictionary with keys `exists` (list of booleans), `counts` (list of instance counts per image), and `summary` (a short text summary).

PerFrameMask object. The returned object exposes `frame_indices`, `labels`, `num_frames`, and

`seg.get_mask(frame=fi, object=k)`: 2D boolean mask of object k (k may be an index or a string label).

`seg[fi]`: (N_{obj} , H , W) boolean stack at frame fi .

`seg.get_centroid_3d(recon, frame=fi, object=k)`: median 3D world position of the masked points, or `None`.

`seg.get_masked_points(recon, frame=fi, object=k)`: (K , 3) array of masked world points.

`seg.visualize(fi)`: `VisualFeedback` overlay for verification with `show()`.

G.3 `tools.Geometry`

`tools.Geometry` is a static class that provides the numerical primitives most commonly needed for spatial reasoning over reconstructed scenes. All methods operate on `numpy` arrays.

Methods.

- `euclidean_distance(p1, p2)`
 - *Input*: two 1D arrays of length 3.
 - *Output*: `float`, the Euclidean distance.

`angle_between_vectors(v1, v2)`

Input: two 3D vectors.

Output: `float`, the angle in degrees.

`project_point_to_camera(point_3d, c2w, fx, fy, cx, cy)`

Input: a 3D world point, a 4×4 camera-to-world `SE(3)` matrix, and the four scalar intrinsics.

Output: `(u, v)` pixel coordinates, or `None` when the point is behind the camera.

`rotation_matrix_from_vectors(v_from, v_to)`

Input: two 3D vectors (need not be unit length).

Output: a 3×3 rotation matrix that aligns `v_from` with `v_to`.

`transform_points(points, matrix)`

Input: a `(..., 3)` array of points and a 4×4 `SE(3)` matrix.

Output: a transformed array with the same shape as the input.

`fit_ground_plane_ransac(points, confidence, conf_threshold = 0.3, n_iterations = 1000, inlier_threshold = 0.05)`

Input: a (H, W, 3) or (N, 3) point cloud and a matching confidence array; thresholds and iteration count are optional.

Output: (plane_normal, inlier_mask) via RANSAC [10], or (None, None) when fitting fails. The sign of plane_normal is unspecified and is disambiguated by the caller.

normalized_{top}pixel(coords, width, height)

Input: a sequence of coordinates in the 0–1000 normalized scale used by vlm.locate, together with the image width and height in pixels.

Output: a list[float] of pixel coordinates.

G.4 tools.Mask

tools.Mask is a static class that provides standard mask statistics over the boolean mask arrays returned by tools.SAM3. All inputs are 2D numpy arrays with shape (H, W) unless otherwise noted.

Methods.

- centroid(mask)
 - *Input:* a 2D boolean mask.
 - *Output:* (cx, cy) pixel coordinates of the median; (nan, nan) when the mask is empty.
- centroids(masks)
 - *Input:* an (N, H, W)

batch of boolean masks.

- *Output:* a list of `(cx, cy)` centroids, one per element of the batch.

- `area(mask)`

- *Input:* a 2D boolean mask.
- *Output:* `int`, the number of `True` pixels.

- `bounding_box(mask)`

- *Input:* a 2D boolean mask.
- *Output:* `(x1, y1, x2, y2)` in pixels, or `None` when the mask is empty. For masks with more than 100 `True` pixels, the box is computed from the 1st and 99th percentiles to discard outliers.

`iou(mask_a, mask_b)`

- *Input:* two boolean masks of identical shape.
- *Output:* `float`, the intersection over union.

`intersection(mask_a, mask_b)`

- *Input:* two boolean masks of identical shape.
- *Output:* a boolean mask of the same shape, the element-wise AND.

`mask_to_box(mask)`

Input: a 2D boolean mask.

Output: a 4-element `numpy` array `[x1, y1, x2, y2]`, or `None` when the mask is empty.

G.5 `tools.Time`

`tools.Time` is a utility that converts between frame indices and seconds. It is initialized from the input metadata's frame rate and total frame count, and is meaningful only when `Metadata.is_video` is true and `Metadata.fps` is set.

Methods.

- `frametoseconds(frameiindex)`

- *Input:* an integer or float frame index.
- *Output:* `float`, the corresponding time in seconds.

`secondstoframe(seconds)`

Input: a number of seconds.

Output: `int`, the nearest frame index, clamped to the valid range.

`framerangetoseconds(startfframe, endfframe)`

Input: two frame indices.

Output: `float`, the duration

in seconds between them.

get_{frame}_{at}_{time}(seconds) :
aliasforseconds_to_frame.

G.6 tools.Graph

tools.Graph is a utility that wraps `matplotlib` to produce diagnostic line plots from numerical sequences. The plot is returned as a `VisualFeedback` that the agent inspects inline with `show()` and that also carries a short text summary (minimum, maximum, mean, and trend) accessible as `chart.description`.

Method.

- `plot(values, validity=None, xlabel = "Frame", ylabel = "Value", title = None)`

- *Input:*

`values`
is a 1D
`numpy`
ar-
ray of
length T .
`validity`
is an
op-
tional
1D
boolean
array
of the
same
length
whose
`False`
entries
are ren-
dered
as gaps.
`x_label`,
`y_label`,
and
`title`
are dis-
play
strings.

- *Output:* a `VisualFeedback` con-
taining
the ren-
dered
plot
and a
text de-
scrip-
tion.

G.7 `tools.Draw`

`tools.Draw` is a static class that produces PIL-based annotation overlays. All methods accept a `numpy (H, W, 3) uint8` array, a PIL image, or an `InputImages[i]` entry, and return a PIL image of the same size; the input image is never modified. Colours may be passed as an `(R, G, B) uint8` tuple or as any `matplotlib` or CSS4 colour name. Each method accepts either a single primitive or a list of primitives, and a matching list of colours when per-primitive colouring is desired.

Methods.

- `draw_box(image, bboxes, colors)`

*None, thickness =
None)*

- *Input:*
bboxes
is a
single
(x1,
y1,
x2,
y2) in
pixels
or a list
of such
boxes.

- *Output:*
a PIL
image
with
axis-
aligned
rectan-
gle out-
lines
drawn.

*draw_line(image, lines, colors =
None, thickness =
None)*

- *Input:*
lines
is a sin-
gle (x1,
y1, x2,
y2) in
pixels or
a list of
such seg-
ments.

- *Output:*
a PIL

image
with
line seg-
ments
drawn.

```
draw_point(image, points, co  
None, radius =  
None)
```

In-
put:
`points`
is
a
sin-
gle
(`x`,
`y`)
in
pix-
els
or
a
list
of
such
points.

Out-
put:
a
PIL
im-
age
with
filled
cir-
cles
drawn.

H
Lim-
i-
ta-
tions
and
Broader
Im-
pact

H.1
Lim-
i-
ta-
tions

The
main
re-
main-
ing
bot-
tle-
neck
of
Spa-
tial-
Claw
is
the
per-
cep-
tual
qual-
ity
of
the
back-
bone
vision-

language
model
and
of
the
per-
cep-
tion
tools
it
com-
poses.
The
failure-
mode
anal-
y-
sis
in
the
main
pa-
per
at-
tributes
the
largest
share
of
re-
main-
ing
er-
rors
to
per-
cep-
tion
rather
than
to
the

ac-
tion
in-
ter-
face,
which
means
that
fur-
ther
in-
ter-
face
de-
sign
has
di-
min-
ish-
ing
re-
turns
at
the
cur-
rent
scale
of
eval-
ua-
tion.
We
ex-
pect
the
prin-
ci-
pal
axis
for
fu-
ture

im-
prove-
ment
to
be
per-
cep-
tual
qual-
ity,
both
in
the
back-
bone
vision-
language
model
and
in
the
un-
der-
ly-
ing
re-
con-
struc-
tion
and
seg-
men-
ta-
tion
mod-
els.

H.2

Broader

Im-

pact

Spa-

tial-

Claw

is

training-

free

and

adds

no

pa-

ram-

e-

ters

to

the

back-

bone

vision-

language

model.

Ex-

ist-

ing

mod-

els

can

there-

fore

be

ex-

tended

with

stronger

spa-

tial

rea-

son-

ing
abil-
ity
with-
out
ad-
di-
tional
train-
ing
data
and
with-
out
fine-
tuning,
which
is
par-
tic-
u-
larly
valu-
able
in
do-
mains
where
data
col-
lec-
tion
is
ex-
pen-
sive
or
im-
prac-
ti-
cal,
in-

clud-
ing
robotics,
em-
bod-
ied
ap-
pli-
ca-
tions,
and
as-
sis-
tive
sys-
tems.
Be-
cause
the
same
con-
fig-
u-
ra-
tion
trans-
fers
across
back-
bones
and
bench-
marks
with-
out
mod-
ifi-
ca-
tion,
the
frame-
work

in-
creases
the
prac-
ti-
cal
value
of
vision-
language
mod-
els
that
are
al-
ready
de-
ployed.

Ref-
er-
ences

[1]
El-
lis
Brown,
Ji-
han
Yang,
Shusheng
Yang,
Rob
Fer-
gus,
and
Sain-
ing
Xie.
Bench-
mark

de-
sign-
ers
should
“train
on
the
test
set”
to
ex-
pose
ex-
ploitable
non-
visual
short-
cuts.
*ArXiv
Preprint,*
2025.

[2]

Nico-
las
Car-
ion,
Fran-
cisco
Massa,
Gabriel
Syn-
naeve,
Nico-
las
Usunier,
Alexan-
der
Kir-
illov,
and
Sergey

Zagoruyko.
End-
to-
end
ob-
ject
de-
tec-
tion
with
trans-
form-
ers.
In
*Eu-
ro-
pean
Con-
fer-
ence
on
Com-
puter
Vi-
sion
(ECCV)*,
2020.

[3]

Nico-
las
Car-
ion,
Laura
Gustafson,
Yuan-
Ting
Hu,
Shoub-
hik
Deb-
nath,

Rong-
hang
Hu,
Di-
dac
Suris,
Chai-
tanya
Ryali,
Kalyan Va-
sudev
Al-
wala,
Haitham
Khedr,
An-
drew
Huang,
et al.
Sam
3:
Seg-
ment
any-
thing
with
con-
cepts.
In
*In-
ter-
na-
tional
Con-
fer-
ence
on
Learn-
ing
Rep-
re-
sen-*

*ta-
tions
(ICLR),
2026.*

[4]

Boyuan
Chen,
Zhuo
Xu,
Sean
Kir-
mani,
Brain
Ichter,
Dorsa
Sadigh,
Leonidas
Guibas,
and
Fei
Xia.
Spa-
tialvlm:
En-
dow-
ing
vision-
language
mod-
els
with
spa-
tial
rea-
son-
ing
ca-
pa-
bil-
i-
ties.

In
IEEE
Con-
fer-
ence
on
Com-
puter
Vi-
sion
and
Pat-
tern
Recog-
ni-
tion
(CVPR),
2024.

[5]

Guo
Chen,
Zhiqi
Li,
Shi-
hao
Wang,
Jin-
dong
Jiang,
Yicheng
Liu,
Li-
dong
Lu,
De-
An
Huang,
Won-
min
Byeon,
Matthieu

Le,
Max
Ehrlich,
Tong
Lu,
Limin
Wang,
Bryan
Catan-
zaro,
Jan
Kautz,
An-
drew
Tao,
Zhid-
ing
Yu,
and
Guilin
Liu.
Ea-
gle
2.5:
Boost-
ing
long-
context
post-
training
for
fron-
tier
vision-
language
mod-
els.
In
*Ad-
vances
in
Neu-*

ral
In-
for-
ma-
tion
Pro-
cess-
ing
Sys-
tems
(NeurIPS),
2025.

[6]
Siyi
Chen,
Mikaela An-
gelina
Uy,
Chan Hee
Song,
Faisal
Lad-
hak,
Adithyavaira-
van
Mu-
rali,
Qing
Qu,
Stan
Birch-
field,
Valts
Blukis,
and
Jonathan
Trem-
blay.
Space-
tools:
Tool-

augmented
spa-
tial
rea-
son-
ing
via
dou-
ble
in-
ter-
ac-
tive
rl.
In
*IEEE
Con-
fer-
ence
on
Com-
puter
Vi-
sion
and
Pat-
tern
Recog-
ni-
tion
(CVPR),
2026.*

[7]
An-
Chieh
Cheng,
Hongxu
Yin,
Yang
Fu,
Qiushan

Guo,
Rui-
han
Yang,
Jan
Kautz,
Xi-
ao-
long
Wang,
and
Sifei
Liu.
Spa-
tial-
rgpt:
Grounded
spa-
tial
rea-
son-
ing
in
vision-
language
mod-
els.
In
*Ad-
vances
in
Neu-
ral
In-
for-
ma-
tion
Pro-
cess-
ing
Sys-
tems*

(*NeurIPS*),
2024.

[8] Seokju
Cho,
Ab-
hishek
Badki,
Hang
Su,
Jin-
dong
Jiang,
Ziyao
Zeng,
Se-
un-
gry-
ong
Kim,
Sifei
Liu,
and
Orazio
Gallo.
4dp-
qa:
Scal-
able
qa
for
4d
per-
cep-
tion
in
vi-
sion
lan-
guage
mod-

els.
In
*IEEE
Con-
fer-
ence
on
Com-
puter
Vi-
sion
and
Pat-
tern
Recog-
ni-
tion
(CVPR)*,
2026.

[9]

Zhi-
wen
Fan,
Jian
Zhang,
Ren-
jie
Li,
Junge
Zhang,
Run-
jin
Chen,
Hezhen
Hu,
Kevin
Wang,
Huaizhi
Qu,
Dilin
Wang,

Zhicheng
Yan,
et al.
Vlm-
3r:
Vision-
language
mod-
els
aug-
mented
with
instruction-
aligned
3d
re-
con-
struc-
tion.
In
*IEEE
Con-
fer-
ence
on
Com-
puter
Vi-
sion
and
Pat-
tern
Recog-
ni-
tion
(CVPR)*,
2026.

[10]

Mar-
tin A
Fis-

chler
and
Robert C
Bolles.
Ran-
dom
sam-
ple
con-
sen-
sus:
a
paradigm
for
model
fit-
ting
with
ap-
pli-
ca-
tions
to
im-
age
anal-
y-
sis
and
au-
to-
mated
car-
tog-
ra-
phy.
*Com-
mu-
ni-
ca-
tions
of*

the
ACM,
24
(6):
381–395,
1981.

[11]

Elias
Fran-
tar,
Saleh
Ashk-
boos,
Torsten
Hoe-
fler,
and
Dan
Al-
is-
tarh.
GPTQ:
Ac-
cu-
rate
post-
training
com-
pres-
sion
for
gen-
er-
a-
tive
pre-
trained
trans-
form-
ers.
ArXiv

Preprint,
2022.

[12] Chaoyou
Fu,
Yuhan
Dai,
Yong-
dong
Luo,
Lei
Li,
Shuhuai
Ren,
Ren-
rui
Zhang,
Zi-
han
Wang,
Chenyu
Zhou,
Yun-
hang
Shen,
Meng-
dan
Zhang,
et al.
Video-
mme:
The
first-
ever
com-
pre-
hen-
sive
eval-
u-
a-

tion
bench-
mark
of
multi-
modal
llms
in
video
anal-
y-
sis.
In
*IEEE
Con-
fer-
ence
on
Com-
puter
Vi-
sion
and
Pat-
tern
Recog-
ni-
tion
(CVPR)*,
2025.

[13]
Xingyu
Fu,
Yushi
Hu,
Bangzheng
Li,
Yu Feng,
Haoyu
Wang,
Xudong

Lin,
Dan
Roth,
Noah A
Smith,
Wei-
Chiu
Ma,
and
Ran-
jay
Kr-
ishna.
Blink:
Mul-
ti-
modal
large
lan-
guage
mod-
els
can
see
but
not
per-
ceive.
In
*Eu-
ro-
pean
Con-
fer-
ence
on
Com-
puter
Vi-
sion
(ECCV)*,
2024.

[14]

Google
Deep-
Mind.
Gemma
4.
[https://
deepmind.
google/
models/
gemma/
gemma-4/](https://deepmind.google/models/gemma/gemma-4/),
2026.
Ac-
cessed:
2026-
04-
14.

[15]

Tan-
may
Gupta
and
Anirud-
dha
Kem-
b-
havi.
Vi-
sual
pro-
gram-
ming:
Com-
po-
si-
tional
vi-
sual
rea-

son-
ing
with-
out
train-
ing.
In
*IEEE
Con-
fer-
ence
on
Com-
puter
Vi-
sion
and
Pat-
tern
Recog-
ni-
tion
(CVPR),
2023.*

[16]
Yi Han,
En-
shen
Zhou,
Shanyu
Rong,
Jingkun
An,
Peng-
wei
Wang,
Zhongyuan
Wang,
Cheng
Chi,
Lu Sheng,

and
Shang-
hang
Zhang.
Tiger:
Tool-
integrated
ge-
o-
met-
ric
rea-
son-
ing
in
vision-
language
mod-
els
for
robotics.
*ArXiv
Preprint*,
2025.

[17] Charles R.
Har-
ris,
K. Jar-
rod
Mill-
man,
Sté-
fan J.
van der
Walt,
Ralf
Gom-
mers,
Pauli
Vir-

ta-
nen,
David
Cour-
na-
peau,
Eric
Wieser,
Ju-
lian
Tay-
lor,
Se-
bas-
tian
Berg,
Nathaniel J.
Smith,
Robert
Kern,
Matti
Pi-
cus,
Stephan
Hoyer,
Marten H.
van
Kerk-
wijk,
Matthew
Brett,
Al-
lan
Hal-
dane,
Jaime Fer-
nán-
dez
del
Río,
Mark
Wiebe,

Pearu
Pe-
ter-
son,
Pierre
Gérard-
Marchant,
Kevin
Shep-
pard,
Tyler
Reddy,
War-
ren
Weckesser,
Hameer
Ab-
basi,
Christoph
Gohlke,
and
Travis E.
Oliphant.
Ar-
ray
pro-
gram-
ming
with
NumPy.
*Na-
ture*,
585
(7825):
357–362,
Septem-
ber
2020.
doi:
10.
1038/
s41586-020-2649-

URL
<https://doi.org/10.1038/s41586-020-264>

[18] Wenbo Hu, Jingli Lin, Yilin Long, Yunlong Ran, Lihan Jiang, Yifan Wang, Chenming Zhu, Runsen Xu, Tai Wang, and Jiangmiao Pang. G²vlm: Geom-e-try

grounded
vi-
sion
lan-
guage
model
with
uni-
fied
3d
re-
con-
struc-
tion
and
spa-
tial
rea-
son-
ing.
In
*IEEE
Con-
fer-
ence
on
Com-
puter
Vi-
sion
and
Pat-
tern
Recog-
ni-
tion
(CVPR)*,
2026.

[19]

J. D.
Hunter.

Mat-
plotlib:
A
2d
graph-
ics
en-
vi-
ron-
ment.
*Com-
put-
ing
in
Sci-
ence
’
En-
gi-
neer-
ing,*
9
(3):
90–95,
2007.
doi:
10.
1109/
MCSE.
2007.
55.

[20]
Mengdi
Jia,
Zekun
Qi,
Shaochen
Zhang,
Wenyao
Zhang,
Xin-

qiang
Yu,
Ji-
awei
He,
He Wang,
and
Li Yi.
Om-
nis-
pa-
tial:
To-
wards
com-
pre-
hen-
sive
spa-
tial
rea-
son-
ing
bench-
mark
for
vi-
sion
lan-
guage
mod-
els.
In
*In-
ter-
na-
tional
Con-
fer-
ence
on
Learn-*

ing
Rep-
re-
sen-
ta-
tions
(ICLR),
2026.

[21]
Woosuk
Kwon,
Zhuo-
han
Li,
Siyuan
Zhuang,
Ying
Sheng,
Lian-
min
Zheng,
Cody Hao
Yu,
Joseph E.
Gon-
za-
lez,
Hao
Zhang,
and
Ion
Sto-
ica.
Ef-
fi-
cient
mem-
ory
man-
age-
ment

for
large
lan-
guage
model
serv-
ing
with
page-
dat-
ten-
tion.
In
*Pro-
ceed-
ings
of
the
ACM
SIGOPS
29th
Sym-
po-
sium
on
Op-
er-
at-
ing
Sys-
tems
Prin-
ci-
ples,*
2023.

[22]

Ding-
ming
Li,
Hongx-
ing

Li,
Zix-
uan
Wang,
Yuchen
Yan,
Hang
Zhang,
Siqu
Chen,
Guiyang
Hou,
Sheng-
pei
Jiang,
Wenqi
Zhang,
Yongliang
Shen,
Weim-
ing
Lu,
and
Yuet-
ing
Zhuang.
Viewspatial-
bench:
Eval-
u-
at-
ing
multi-
perspective
spa-
tial
lo-
cal-
iza-
tion
in
vision-

language
mod-
els.
ArXiv
Preprint,
2025.

[23] Shaox-
uan
Li,
Zhix-
uan
Zhao,
Hanze
Deng,
Zirun
Ma,
Shulin
Tian,
Zuyan
Liu,
Yushi
Hu,
Haon-
ing
Wu,
Yuhao
Dong,
Ben-
lin
Liu,
Zi-
wei
Liu,
and
Ran-
jay
Kr-
ishna.
Per-
cep-

tion-
Comp:
A
video
bench-
mark
for
com-
plex
perception-
centric
rea-
son-
ing.
*ArXiv
Preprint*,
2026.

[24]
Hao-
tong
Lin,
Sili
Chen,
Jun Hao
Liew,
Donny Y.
Chen,
Zhenyu
Li,
Guang
Shi,
Ji-
ashi
Feng,
and
Bingyi
Kang.
Depth
any-
thing
3:

Re-
cov-
er-
ing
the
vi-
sual
space
from
any
views.
In
*In-
ter-
na-
tional
Con-
fer-
ence
on
Learn-
ing
Rep-
re-
sen-
ta-
tions
(ICLR)*,
2026.

[25]

Jingli
Lin,
Run-
sen
Xu,
Shao-
hao
Zhu,
Si-
han
Yang,

Peizhou
Cao,
Yun-
long
Ran,
Miao
Hu,
Chen-
ming
Zhu,
Yi-
man
Xie,
Yilin
Long,
Wenbo
Hu,
Dahua
Lin,
Tai
Wang,
and
Jiang-
miao
Pang.
Mmsi-
video-
bench:
A
holis-
tic
bench-
mark
for
video-
based
spa-
tial
in-
tel-
li-
gence.

*ArXiv
Preprint,
2025.*

[26]

Pan
Lu,
Bowen
Chen,
Sheng
Liu,
Rahul
Thapa,
Joseph
Boen,
and
James
Zou.
Oc-
to-
tools:
An
agen-
tic
frame-
work
with
ex-
ten-
si-
ble
tools
for
com-
plex
rea-
son-
ing.
In
*Pro-
ceed-
ings*

*of
the
As-
so-
ci-
a-
tion
for
Com-
pu-
ta-
tional
Lin-
guis-
tics
(ACL),
2026.*

[27]
Zhan-
peng
Luo,
Ce Zhang,
Si-
long
Yong,
Cunxi
Dai,
Qian-
wei
Wang,
Haoxi
Ran,
Guanya
Shi,
Ka-
tia
Sycara,
and
Yaqi
Xie.
pys-

pa-
tial:
Gen-
er-
at-
ing
3d
vi-
sual
pro-
grams
for
zero-
shot
spa-
tial
rea-
son-
ing.
In
*In-
ter-
na-
tional
Con-
fer-
ence
on
Learn-
ing
Rep-
re-
sen-
ta-
tions
(ICLR)*,
2026.

[28]

Dami-
ano
Mar-

sil,
Ro-
hun
Agrawal,
Yisong
Yue,
and
Geor-
gia
Gkioxari.
Vi-
sual
agen-
tic
ai
for
spa-
tial
rea-
son-
ing
with
a
dy-
namic
api.
In
*IEEE
Con-
fer-
ence
on
Com-
puter
Vi-
sion
and
Pat-
tern
Recog-
ni-
tion*

(*CVPR*),
2025.

[29]

Qwen
Team.
Qwen3.5:
To-
wards
na-
tive
mul-
ti-
modal
agents,
Febru-
ary
2026.
URL
[https://
qwen.
ai/
blog?
id=
qwen3.
5.](https://qwen.ai/blog?id=qwen3.5)

[30]

Qwen
Team.
Qwen3.6-
27B:
Flagship-
level
cod-
ing
in
a
27B
dense
model,
April

2026.
URL
[https://
qwen.
ai/
blog?
id=
qwen3.
6-27b.](https://qwen.ai/blog?id=qwen3.6-27b)

[31]

Qwen
Team.
Qwen3.6-
35B-
A3B:
Agen-
tic
cod-
ing
power,
now
open
to
all,
April
2026.
URL
[https://
qwen.
ai/
blog?
id=
qwen3.
6-35b-a3b.](https://qwen.ai/blog?id=qwen3.6-35b-a3b)

[32]

Nikhila
Ravi,
Valentin
Gabeur,

Yuan-
Ting
Hu,
Rong-
hang
Hu,
Chai-
tanya
Ryali,
Tengyu
Ma,
Haitham
Khedr,
Ro-
man
Rä-
dle,
Chloe
Rol-
land,
Laura
Gustafson,
et al.
Sam
2:
Seg-
ment
any-
thing
in
im-
ages
and
videos.
In
*In-
ter-
na-
tional
Con-
fer-
ence*

*on
Learn-
ing
Rep-
re-
sen-
ta-
tions
(ICLR),
2025.*

[33]

Fer-
nando
Roperio,
Erkin
Turkoz,
Daniel
Matos,
Jun-
qing
Du,
An-
to-
nio
Ruiz,
Yan-
feng
Zhang,
Lu Liu,
Ming-
wei
Sun,
and
Yongliang
Wang.
Riemind:
Geometry-
grounded
spa-
tial
agent

for
scene
un-
der-
stand-
ing.
ArXiv
Preprint,
2026.

[34] Yongliang
Shen,
Kaitao
Song,
Xu Tan,
Dong-
sheng
Li,
Weim-
ing
Lu,
and
Yuet-
ing
Zhuang.
Hug-
ging-
gpt:
Solv-
ing
ai
tasks
with
chat-
gpt
and
its
friends
in
hug-
ging

face.
In
Advances
in
Neu-
ral
In-
for-
ma-
tion
Pro-
cess-
ing
Sys-
tems
(*NeurIPS*),
2023.

[35]

Chan Hee
Song,
Valts
Blukis,
Jonathan
Trem-
blay,
Stephen
Tyree,
Yu Su,
and
Stan
Birch-
field.
Ro-
boSpa-
tial:
Teach-
ing
spa-
tial
un-

der-
stand-
ing
to
2D
and
3D
vision-
language
mod-
els
for
robotics.
In
*IEEE
Con-
fer-
ence
on
Com-
puter
Vi-
sion
and
Pat-
tern
Recog-
ni-
tion
(CVPR)*,
2025.

[36]

Dí-
dac
Surís,
Sa-
chit
Menon,
and
Carl
Von-

drick.
Vipergpt:
Vi-
sual
in-
fer-
ence
via
python
ex-
e-
cu-
tion
for
rea-
son-
ing.
In
*IEEE
In-
ter-
na-
tional
Con-
fer-
ence
on
Com-
puter
Vi-
sion
(ICCV)*,
2023.

[37]

Gem-
ini
Team,
Ro-
han
Anil,
Se-

bas-
tian
Borgeaud,
Jean-
Baptiste
Alayrac,
Ji-
ahui
Yu,
Radu
Sori-
cut,
Jo-
han
Schalk-
wyk,
An-
drew M
Dai,
Anja
Hauth,
Katie
Mil-
li-
can,
et al.
Gem-
ini:
a
fam-
ily
of
highly
ca-
pa-
ble
mul-
ti-
modal
mod-
els.
ArXiv

Preprint,
2023.

[38]

Gem-
ini Robotics
Team,
Sa-
minda
Abeyruwan,
Joshua
Ainslie,
Jean-
Baptiste
Alayrac,
Montser-
rat Gon-
za-
lez
Are-
nas,
Travis
Arm-
strong,
Ash-
win
Bal-
akr-
ishna,
Robert
Baruch,
Maria
Bauza,
Michiel
Blokz-
ijl,
et al.
Gem-
ini
robotics:
Bring-
ing

ai
into
the
phys-
i-
cal
world.
ArXiv
Preprint,
2025.

[39]

Video-
MME
Team.
Video-
mme-
v2:
Eval-
u-
at-
ing
true
un-
der-
stand-
ing
and
rea-
son-
ing
in
video
mllms.
ArXiv
Preprint,
2026.
URL
[https://
github.
com/](https://github.com/)

Video-MME/
Video-MME-v2.

[40] Pauli
Vir-
ta-
nen,
Ralf
Gom-
mers,
Travis E.
Oliphant,
Matt
Haber-
land,
Tyler
Reddy,
David
Cour-
na-
peau,
Ev-
geni
Burovski,
Pearu
Pe-
ter-
son,
War-
ren
Weckesser,
Jonathan
Bright,
Sté-
fan J.
van
der
Walt,
Matthew
Brett,
Joshua

Wil-
son,
K. Jar-
rod
Mill-
man,
Niko-
lay
May-
orov,
An-
drew
R. J.
Nel-
son,
Eric
Jones,
Robert
Kern,
Eric
Lar-
son,
C J
Carey,
il-
han
Po-
lat,
Yu Feng,
Eric W.
Moore,
Jake
Van-
der-
Plas,
De-
nis
Lax-
alde,
Josef
Perk-
told,

Robert
Cim-
r-
man,
Ian
Hen-
rik-
sen,
E. A.
Quin-
tero,
Charles R.
Har-
ris,
Anne M.
Archibald,
An-
tônio H.
Ribeiro,
Fabian
Pe-
dregosa,
Paul
van
Mul-
bregt,
and
SciPy
1.0
Con-
trib-
u-
tors.
SciPy
1.0:
Fun-
da-
men-
tal
Al-
go-
rithms

for
Sci-
en-
tific
Com-
put-
ing
in
Python.
*Na-
ture
Meth-
ods*,
17:
261–272,
2020.
doi:
10.
1038/
s41592-019-0686-

[41]

Qi-
neng
Wang,
Baiqiao
Yin,
Pingyue
Zhang,
Jian-
shu
Zhang,
Kan-
grui
Wang,
Zi-
han
Wang,
Jieyu
Zhang,
Keshigeyan
Chan-

drasegaran,
Han
Liu,
Ran-
jay
Kr-
ishna,
Sain-
ing
Xie,
Man-
ling
Li,
Ji-
a-
jun
Wu,
and
Li Fei-
Fei.
Spa-
tial
men-
tal
mod-
el-
ing
from
lim-
ited
views.
*ArXiv
Preprint*,
2025.

[42]
Xingyao
Wang,
Yangyi
Chen,
Li-
fan

Yuan,
Yizhe
Zhang,
Yun-
zhu
Li,
Hao
Peng,
and
Heng
Ji.
Ex-
e-
cutable
code
ac-
tions
elicit
bet-
ter
llm
agents.
In
*In-
ter-
na-
tional
Con-
fer-
ence
on
Ma-
chine
Learn-
ing
(ICML)*,
2024.

[43]

Yi-
fan
Wang,

Jian-
jun
Zhou,
Haoyi
Zhu,
Wen-
zheng
Chang,
Yang
Zhou,
Zizun
Li,
Junyi
Chen,
Jiang-
miao
Pang,
Chun-
hua
Shen,
and
Tong
He.
 π^3 :
Permutation-
equivariant
vi-
sual
ge-
om-
e-
try
learn-
ing.
In
*In-
ter-
na-
tional
Con-
fer-
ence*

*on
Learn-
ing
Rep-
re-
sen-
ta-
tions
(ICLR),
2026.*

[44] Min-
grui
Wu,
Zhaozhi
Wang,
Fangjin-
hua
Wang,
Jiao-
long
Yang,
Marc
Polle-
feys,
and
Tong
Zhang.
From
in-
door
to
open
world:
Re-
veal-
ing
the
spa-
tial
rea-

son-
ing
gap
in
mllms.
*ArXiv
Preprint*,
2025.

[45]

Yuxi
Xiao,
Longfei
Li,
Shen
Yan,
Xin-
hang
Liu,
Sida
Peng,
Yun-
chao
Wei,
Xi-
aowei
Zhou,
and
Bingyi
Kang.
Spa-
tial-
tree:
How
spa-
tial
abil-
i-
ties
branch
out
in

mlms.
ArXiv
Preprint,
2025.

[46]

Peiran
Xu,
Sudong
Wang,
Yao
Zhu,
Jian-
ing
Li,
and
Yun-
jian
Zhang.
Spa-
tial-
bench:
Bench-
mark-
ing
mul-
ti-
modal
large
lan-
guage
mod-
els
for
spa-
tial
cog-
ni-
tion.
ArXiv
Preprint,
2025.

[47]

Ji-
han
Yang,
Shusheng
Yang,
An-
jali
Gupta,
Ri-
lyn
Han,
Li Fei-
Fei,
and
Sain-
ing
Xie.
Think-
ing
in
Space:
How
Mul-
ti-
modal
Large
Lan-
guage
Mod-
els
See,
Re-
mem-
ber
and
Re-
call
Spaces.
In
*IEEE
Con-*

*fer-
ence
on
Com-
puter
Vi-
sion
and
Pat-
tern
Recog-
ni-
tion
(CVPR),
2025.*

[48]

Si-
han
Yang,
Run-
sen
Xu,
Yi-
man
Xie,
Sizhe
Yang,
Mo Li,
Jingli
Lin,
Chen-
ming
Zhu,
Xi-
aochen
Chen,
Haodong
Duan,
Xi-
angyu
Yue,

Dahua
Lin,
Tai
Wang,
and
Jiang-
miao
Pang.
Mmsi-
bench:
A
bench-
mark
for
multi-
image
spa-
tial
in-
tel-
li-
gence.
In
ICLR,
2025.

[49]

Chen
Zeren,
Lu Xi-
aoya,
Zheng
Zhi-
jie,
Li Pen-
grui,
He Lehan,
Zhou
Yi-
jin,
Shao
Jing,

Zhuang
Bo-
han,
and
Sheng
Lu.
Geometrically-
constrained
agent
for
spa-
tial
rea-
son-
ing.
*ArXiv
Preprint*,
2025.

[50]
Ji-
ahui
Zhang,
Yu-
rui
Chen,
Yan-
peng
Zhou,
Yuem-
ing
Xu,
Ze Huang,
Jilin
Mei,
Jun-
hui
Chen,
Yu-
jie
Yuan,
Xinyue

Cai,
Guowei
Huang,
Xingyue
Quan,
Hang
Xu,
and
Li Zhang.
From
flat-
land
to
space:
Teach-
ing
vision-
language
mod-
els
to
per-
ceive
and
rea-
son
in
3d.
In
*Ad-
vances
in
Neu-
ral
In-
for-
ma-
tion
Pro-
cess-
ing
Sys-*

tems
(NeurIPS),
2025.

[51]
Shi-
hua
Zhang,
Qi-
uhong
Shen,
Shizun
Wang,
Tianbo
Pan,
and
Xin-
chao
Wang.
Make
ge-
om-
e-
try
mat-
ter
for
spa-
tial
rea-
son-
ing.
ArXiv
Preprint,
2026.

[52]
Za-
ibin
Zhang,
Yuhan
Wu,
Lian-

jie
Jia,
Yi-
fan
Wang,
Zhongbo
Zhang,
Yi-
jiang
Li,
Bing-
hao
Ran,
Fuxi
Zhang,
Zhuo-
han
Sun,
Zhen-
fei
Yin,
et al.
Think3d:
Think-
ing
with
space
for
spa-
tial
rea-
son-
ing.
*ArXiv
Preprint*,
2026.

[53]
Ziang
Zhang,
Ze-
han

Wang,
Guang-
hao
Zhang,
Wei-
long
Dai,
Yan
Xia,
Ziang
Yan,
Min-
jie
Hong,
and
Zhou
Zhao.
Dsi-
bench:
A
bench-
mark
for
dy-
namic
spa-
tial
in-
tel-
li-
gence.
*ArXiv
Preprint*,
2025.

[54]

Shi-
tian
Zhao,
Hao-
quan
Zhang,

Shao-
heng
Lin,
Ming
Li,
Qi-
long
Wu,
Kaipeng
Zhang,
and
Chen
Wei.
Pyvi-
sion:
Agen-
tic
vi-
sion
with
dy-
namic
tool-
ing.
*ArXiv
Preprint*,
2025.

[55]

Lian-
min
Zheng,
Wei-
Lin
Chi-
ang,
Ying
Sheng,
Siyuan
Zhuang,
Zhang-
hao

Wu,
Yong-
hao
Zhuang,
Zi Lin,
Zhuo-
han
Li,
Dacheng
Li,
Eric
Xing,
et al.
Judg-
ing
llm-
as-
a-
judge
with
mt-
bench
and
chat-
bot
arena.
In
*Ad-
vances
in
Neu-
ral
In-
for-
ma-
tion
Pro-
cess-
ing
Sys-
tems
(NeurIPS)*,

2023.

[56]

Fengzhe
Zhou,
Jian-
nan
Huang,
Jialuo
Li,
Deva
Ra-
manan,
and
Humphrey
Shi.
Pai-
bench:
A
com-
pre-
hen-
sive
bench-
mark
for
phys-
i-
cal
ai.
*ArXiv
Preprint*,
2025.

[57]

Nan-
nan
Zhu,
Yong-
hao
Dong,
Teng
Wang,

Xue-
qian
Li,
Shengjun
Deng,
Yi-
jia
Wang,
Zheng
Hong,
Tiantian
Geng,
Guo
Niu,
Hanyan
Huang,
et al.
Cvbench:
Bench-
mark-
ing
cross-
video
syn-
er-
gies
for
com-
plex
mul-
ti-
modal
rea-
son-
ing.
*ArXiv
Preprint*,
2025.

Model	Method	Single-image spatial reasoning				Multi-view spatial reasoning			General spatial reasoning			
		ERQA [2023]	Omni3D [2025]	OmniSpatial [2026]	SPBench [2025]	MindCube [2025]	MMSI [2025]	SPAR-Bench [2025]	BLINK [2024]	SpatialTree [2025]	ViewSpatial [2025]	
Qwen3.5-397B-A17B [29]	No-tool	63.0	57.0	62.1	54.0	58.3	46.2	57.2	72.9	65.1	57.3	
	SpatialClaw	62.0 (−1.0)	55.1 (−1.9)	62.8 (+0.7)	63.3 (+9.3)	66.2 (+7.9)	49.7 (+3.5)	66.1 (+8.9)	76.7 (+3.8)	60.0 (−5.1)	57.3 (+0.0)	
Qwen3.5-122B-A10B [29]	No-tool	61.0	55.1	43.3	52.2	49.7	42.7	52.6	73.2	61.6	52.8	
	SpatialClaw	57.5 (−3.5)	52.3 (−2.8)	61.9 (+18.6)	63.6 (+11.4)	61.0 (+11.3)	45.3 (+2.6)	57.6 (+5.0)	74.6 (+1.4)	57.8 (−3.8)	54.5 (+1.7)	
Qwen3.6-35B-A3B [31]	No-tool	57.2	51.2	57.0	52.4	45.4	38.0	47.8	70.6	58.3	52.0	
	SpatialClaw	58.2 (+1.0)	54.0 (+2.8)	59.9 (+2.9)	63.0 (+10.6)	61.9 (+16.5)	45.9 (+7.9)	58.4 (+10.6)	74.1 (+3.5)	60.3 (+2.0)	52.9 (+0.9)	
Qwen3.6-27B [30]	No-tool	59.5	55.9	60.8	58.0	51.2	40.9	55.0	70.5	62.6	53.9	
	SpatialClaw	61.5 (+2.0)	57.7 (+1.8)	62.8 (+2.0)	64.8 (+6.8)	70.1 (+18.9)	53.9 (+13.0)	68.1 (+13.1)	77.4 (+6.9)	64.6 (+2.0)	59.1 (+5.2)	
Gemma4-31B [14]	No-tool	58.3	51.7	57.3	55.1	57.5	37.9	55.2	75.7	59.9	51.7	
	SpatialClaw	61.3 (+3.0)	54.3 (+2.6)	63.6 (+6.3)	68.4 (+13.3)	72.8 (+15.3)	51.3 (+13.4)	63.3 (+8.1)	73.4 (−2.3)	60.7 (+0.8)	60.2 (+8.5)	
Gemma4-26B-A4B [14]	No-tool	56.0	44.8	56.9	45.1	47.8	31.5	48.3	70.2	53.7	53.1	
	SpatialClaw	56.0 (+0.0)	48.9 (+4.1)	57.1 (+0.2)	61.9 (+16.8)	64.0 (+16.2)	42.7 (+11.2)	63.1 (+14.8)	71.5 (+1.3)	57.9 (+4.2)	58.6 (+5.5)	
Model	Method	Video spatial & 4D reasoning						General video understanding				Average
		MMSI-Video [2025]	OSI-Bench [2025]	PAT-Bench [2025]	VSI-Bench-U [2025]	VSTI-Bench [2026]	DST-Bench [2025]	CV-Bench [2025]	PerceptComp [2026]	Video-MME [2025]	Video-MME-v2 [2026]	
Qwen3.5-397B-A17B [29]	No-tool	40.8	37.3	73.3	58.8	60.4	49.4	71.7	42.1	77.5	41.1	57.3
	SpatialClaw	43.6 (+2.8)	45.6 (+8.3)	71.6 (−1.7)	56.6 (−2.2)	67.5 (+7.1)	65.8 (+16.4)	72.9 (+1.2)	43.0 (+0.9)	77.0 (−0.5)	45.7 (+4.6)	60.4 (+3.1)
Qwen3.5-122B-A10B [29]	No-tool	37.9	37.1	69.8	55.7	54.7	47.5	72.1	41.6	75.8	38.2	53.7
	SpatialClaw	38.3 (+0.4)	41.4 (+4.3)	65.6 (−4.2)	50.6 (−5.1)	63.3 (+8.6)	66.4 (+18.9)	72.2 (+0.1)	41.1 (−0.5)	72.6 (−3.2)	40.2 (+2.0)	56.9 (+3.2)
Qwen3.6-35B-A3B [31]	No-tool	35.6	35.5	67.6	54.2	58.8	47.2	71.1	40.4	74.3	37.0	52.6
	SpatialClaw	38.5 (+2.9)	41.9 (+6.4)	63.8 (−3.8)	51.7 (−2.5)	64.2 (+5.4)	64.1 (+16.9)	74.0 (+2.9)	42.0 (+1.6)	75.0 (+0.7)	40.1 (+3.1)	57.2 (+4.6)
Qwen3.6-27B [30]	No-tool	37.1	36.8	69.8	55.4	60.5	48.0	70.6	39.4	76.7	36.7	55.0
	SpatialClaw	47.0 (+9.9)	47.9 (+11.1)	72.1 (+2.3)	57.9 (+2.5)	70.3 (+9.8)	68.1 (+20.1)	75.1 (+4.5)	46.5 (+7.1)	79.4 (+2.7)	49.7 (+13.0)	62.7 (+7.7)
Gemma4-31B [14]	No-tool	36.9	35.6	65.0	48.0	54.7	45.3	69.8	36.8	74.8	40.7	53.4
	SpatialClaw	41.6 (+4.7)	41.9 (+6.3)	68.1 (+3.1)	48.5 (+0.5)	67.6 (+12.9)	62.9 (+17.6)	72.2 (+2.4)	44.0 (+7.2)	77.0 (+2.2)	44.4 (+3.7)	59.9 (+6.5)
Gemma4-26B-A4B [14]	No-tool	32.5	31.1	59.1	30.1	53.1	41.1	66.3	34.8	69.5	35.6	48.0
	SpatialClaw	32.8 (+0.3)	40.0 (+8.9)	59.0 (−0.1)	38.5 (+8.4)	61.7 (+8.6)	61.1 (+20.0)	67.8 (+1.5)	36.3 (+1.5)	69.6 (+0.1)	37.3 (+1.7)	54.3 (+6.3)

TABLE 1 Main results across 20 spatial reasoning benchmarks. For each backbone, we compare a no-tool baseline (thinking mode without tool access) with our SpatialClaw agent framework.

TABLE 2 Action interface comparison. All variants use the same toolset. “Structured Tool-Call” exposes the tools through a JSON command interface [6]. “Single-Pass Code” generates one complete program before execution.

Benchmark	No-tool Baseline	Single-Pass Code	Structured Tool-Call	SpatialClaw (Ours)
<i>Single-image spatial reasoning</i>				
ERQA [38]	58.3	58.3	59.0	61.3
Omni3D [28]	51.7	48.2	55.7	54.3
OmniSpatial [20]	57.3	60.0	59.6	63.6
SPBench [46]	55.1	58.2	60.5	68.4
<i>Multi-view spatial reasoning</i>				
MindCube [41]	57.5	57.5	62.4	72.8
MMSI [48]	37.9	42.3	43.0	51.3
SPAR-Bench [50]	55.2	61.1	58.7	63.3
<i>Video spatial & 4D reasoning</i>				
MMSI-Video [25]	36.9	37.1	35.1	41.6
OSI-Bench [44]	35.6	36.8	40.3	41.9
PAI-Bench [56]	65.0	64.2	65.6	68.1
VSI-Bench-U [1]	48.0	47.8	50.1	48.5
VSTI-Bench [9]	54.7	64.2	63.5	67.6
DSI-Bench [53]	45.3	57.9	58.4	62.9
<i>General spatial reasoning</i>				
BLINK [13]	75.7	73.9	73.9	73.4
SpatialTree [45]	59.9	58.9	57.7	60.7
ViewSpatial [22]	51.7	52.0	55.5	60.2
<i>General video understanding</i>				
CV-Bench [57]	69.8	71.3	73.6	72.2
PerceptComp [23]	36.8	39.2	42.5	44.0
Video-MME [12]	74.8	74.6	75.8	77.0
Video-MME-v2 [39]	40.7	40.2	44.0	44.4
Average (20 bench.)	53.4	55.2	56.7	59.9

TABLE 3 Comparison with other spatial agents. All use the same Gemma4-31B [14] backbone model. *: video or multi-image not supported.

VADAR [28]	pySpatial [27]	SpaceTools [6]	Toolshed	SpatialClaw (Ours)
33.3	50.8	52.0		61.3
44.6	50.6	50.7		54.3
42.4	58.0	60.4		63.6
41.6	53.5	45.1		68.4
—*	67.1	52.9		72.8
—*	33.2	33.1		51.3
—*	51.7	53.9		63.3
—*	28.7	36.6		41.6
—*	32.0	30.2		41.9
—*	46.0	65.1		68.1
—*	44.4	33.6		48.5
—*	55.0	56.0		67.6
—*	43.7	43.0		62.9
—*	60.3	58.2		73.4
—*	53.9	52.7		60.7
—*	49.9	52.1		60.2
—*	67.7	46.8		72.2
—*	37.1	38.6		44.0
—*	50.3	74.6		77.0
—*	23.0	37.4		44.4
—*	47.8	48.7		59.9

Variant	Single-image spatial reasoning				Multi-view spatial reasoning			General spatial reasoning			Video spatial & 4D reasoning				General video understanding	Average
	ERQA [2025]	Omni3D [2025]	OmniSpatial [2026]	SPBench [2025]	MindCube [2025]	MMSI [2025]	SPAR-Bench [2025]	BLINK [2024]	SpatialTree [2025]	ViewSpatial [2025]	OSI-Bench [2025]	PAI-Bench [2025]	VSTI-Bench [2026]	DSI-Bench [2025]	CV-Bench [2025]	Avg.
SpatialClaw (Full)	56.0	36.6	57.2	65.2	67.2	45.1	67.7	71.9	37.8	60.6	44.2	56.2	58.3	61.0	68.9	56.9
(I) No utility functions (e.g., tools.Mask, tools.Geometry)	52.5	37.0	61.6	62.4	69.0	43.1	66.1	71.9	35.1	60.4	42.8	53.0	59.9	62.0	68.7	56.4
(II) No perception tools (no SAM3/DA3)	58.0	33.8	62.2	58.4	49.0	35.5	53.8	70.1	33.1	56.8	34.0	55.2	53.6	48.8	68.9	51.4
No-tool baseline	55.8	32.8	56.6	47.0	47.6	31.5	50.8	69.1	35.5	54.8	35.0	56.0	49.6	43.6	65.4	48.7

TABLE 4 Analysis on tools of SpatialClaw. We ablate two design choices: No utility function, keeping only the perception tools (SAM3/DA3) and removing perception tools and keeping only the utility functions. We additionally report a no-tool baseline (backbone with thinking only). Each variant is evaluated on subsampled examples (500 per benchmark) across 15 benchmarks due to computation constraints. Gemma4-26B-A4B is used as the backbone for its small size.

Benchmark	Category	Per-sample metric
ERQA [38]	Single-image spatial reasoning	Acc
Omni3D [28]	Single-image spatial reasoning	Acc + MRA
OmniSpatial [20]	Single-image spatial reasoning	Acc
SPBench [46]	Single-image spatial reasoning	Acc + MRA
MindCube [41]	Multi-view spatial reasoning	Acc
MMSI [48]	Multi-view spatial reasoning	Acc
SPAR-Bench [50]	Multi-view spatial reasoning	Acc + MRA + VCI
MMSI-Video [25]	Video spatial & 4D reasoning	Acc
OSI-Bench [44]	Video spatial & 4D reasoning	Acc + MRA
PAI-Bench [56]	Video spatial & 4D reasoning	Acc
VSI-Bench-U [1]	Video spatial & 4D reasoning	Acc + MRA
VSTI-Bench [9]	Video spatial & 4D reasoning	Acc + MRA
DSI-Bench [53]	Video spatial & 4D reasoning	Acc
BLINK [13]	General spatial reasoning	Acc
SpatialTree [45]	General spatial reasoning	Acc + MRA
ViewSpatial [22]	General spatial reasoning	Acc
CV-Bench [57]	General video understanding	Acc
PerceptComp [23]	General video understanding	Acc
Video-MME [12]	General video understanding	Acc
Video-MME-v2 [39]	General video understanding	Acc

TABLE 5 Benchmark-specific evaluation metrics and scoring protocols. Each benchmark mixes categorical questions (multiple-choice or binary) and numerical questions in different proportions. **Acc** denotes per-sample 1/0 categorical scoring, **MRA** denotes mean relative accuracy on numerical questions, and **VCI** denotes the SPAR-Bench metric used for the view change inference task (MRA averaged across five movement axes). The reported number for each benchmark is the unweighted mean of the per-sample scores.

Backbone	Size	Quantization	Context
Qwen3.5-397B-A17B [29]	397B (17B activated)	GPTQ Int4 [11]	262K
Qwen3.5-122B-A10B [29]	122B (10B activated)	FP8	262K
Qwen3.6-35B-A3B [31]	35B (3B activated)	FP8	262K
Qwen3.6-27B [30]	27B	FP8	262K
Gemma4-31B [14]	31B	FP8	256K
Gemma4-26B-A4B [14]	26B (4B activated)	FP8	256K

TABLE 6 **Backbones used in the main results.** All six are served through the same vLLM-based [21] serving role and evaluated under identical configuration: the same system prompt, tool set, maximum step count, and input preprocessing across every benchmark.